

DIGITAL COMM, FAST

SAMPLING · QUANTISATION · LINE CODES · INFORMATION THEORY · CARRIER
MODULATION

Twelve chapters, each a screen or two — the night-before version of EC301, not the textbook. Every chapter closes with five lines that are the whole chapter, and the switch in the margin hides everything except those lines.

Read it once end to end, then read only the summaries.

12 CHAPTERS	53 SECTIONS	11 DIAGRAMS	60 SUMMARY LINES
-----------------------	-----------------------	-----------------------	----------------------------

CHAPTER 01

THE WHOLE LINK, IN ONE DIAGRAM

Every other chapter here is a zoom into one box of a single diagram. This chapter puts that diagram on the page and gives each block a one-line job. It also settles the three numbers — BER, SNR and E_b/N_0 — that decide whether a design is any good, and answers the one question examiners ask about this diagram every single year.

1.1 What the system is for

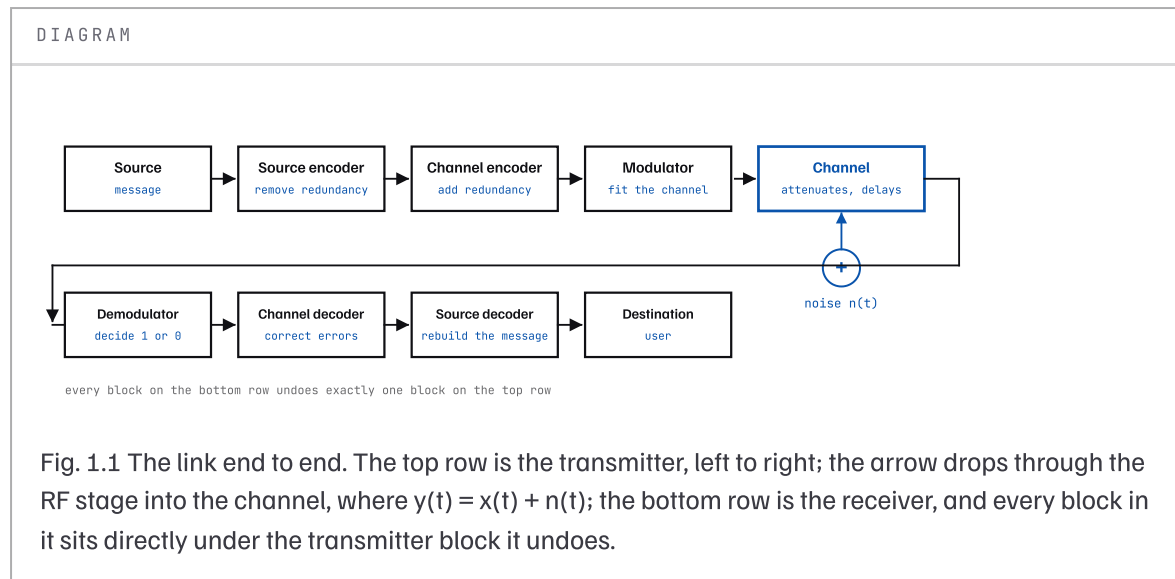
Somebody wants to send a voice, a photo or a sensor reading to somebody far away. In between sits a **channel** — air, copper, fibre, space. The channel weakens the signal, adds noise, bends it out of shape, and lets other people's signals crash into it. The whole subject is one sentence: send information **reliably** and **efficiently** across a channel that is not kind.

WHY GO DIGITAL AT ALL

A damaged analog signal stays damaged, and every amplifier on the route makes it a little worse. A digital signal is only ever 0 or 1, so as long as the noise is not big enough to flip one into the other, the receiver rebuilds it **perfectly**. And because it is only 0s and 1s, you can compress it, encrypt it and add error-correcting maths to it.

1.2 The block diagram

This is worth three marks on its own and it has appeared as a full question. Draw the transmitter row, drop into the channel, draw the receiver row back the other way.



BLOCK	ITS JOB
Information source	Makes the message — microphone, camera, keyboard, sensor.
A/D converter	Sampling, quantisation, encoding. A smooth wave becomes bits. This is Chapter 2.
Source encoder	Squeezes the data. Removes the redundancy already in it. ZIP, JPEG, MP3. Chapter 10.
Channel encoder	Adds designed redundancy so the receiver can spot and repair errors.
Interleaver	Optional. Shuffles bit order, so a burst of damage is spread thinly over many codewords.
Digital modulator	Maps groups of bits onto waveforms. BPSK, QPSK, 16-QAM. Chapter 12.
Up-converter, RF amp	Lifts the signal to the frequency the antenna can radiate, and supplies the power.
Channel	Air, cable, fibre, satellite. Where all the damage happens.
LNA, down-converter	The arriving signal is tiny, so a low-noise amplifier boosts it first, then it is shifted back down.
Demodulator / detector	Decides which symbol was sent. The decision is a guess, and wrong guesses are where bit errors are born.
Channel decoder	Uses the added redundancy to find and fix the wrong guesses.
Source decoder	Un-squeezes back to full size.
D/A converter	Turns numbers back into a wave for the speaker or screen.

THE MIRROR RULE

Every receiver block exactly undoes the matching transmitter block, in reverse order. Source encoder pairs with source decoder, channel encoder with channel decoder, modulator with demodulator. Say this sentence when you finish the diagram — it is a mark on its own.

THE QUESTION THEY ASK EVERY YEAR

“Source coding removes redundancy but channel coding adds it. Is that not a contradiction?” No. Source coding strips out **useless, accidental** redundancy that was already sitting in the data. Channel coding then adds back a **small, mathematically designed** amount, built to fight noise. Throw out the junk, then add the right tool.

1.3 How a system is scored

BER = bits received wrong / total bits sent
 SER = symbols received wrong / total symbols sent
 SNR = signal power / noise power
 $\text{SNR}_{\text{dB}} = 10 \log_{10}(E_b / N_0)$

BER is the report card. A BER of 10^{-6} means one bit in a million comes out wrong. E_b is the energy spent on one bit; N_0 is the noise power in each hertz of bandwidth. So E_b/N_0 asks how loud one bit is compared with the hiss. Higher E_b/N_0 always means lower BER.

1.4 AWGN, four letters

- **Additive** — the noise simply adds: $y(t) = x(t) + n(t)$. It does not multiply or twist the signal.
- **White** — flat across the whole spectrum, like white light containing every colour.
- **Gaussian** — the voltage at any instant follows the bell curve. Small values common, huge values rare.
- **Noise** — unwanted random energy, mostly electrons jiggling because of heat.

CHAPTER 01 AT A GLANCE

- The goal of a digital communication system: send information **reliably** and **efficiently** across a noisy channel.
- The block diagram runs source → A/D → source encoder → channel encoder → modulator → channel → and back the same way in reverse.
- Every receiver block exactly undoes the matching transmitter block — that mirror symmetry is the point of the diagram.
- Source coding removes accidental redundancy; channel coding adds designed redundancy. Not a contradiction — different redundancy.
- Scores: BER, SER, and $\text{SNR}_{\text{dB}} = 10 \log_{10}(E_b/N_0)$. AWGN = additive, white, Gaussian noise, the standard channel model.

CHAPTER 02

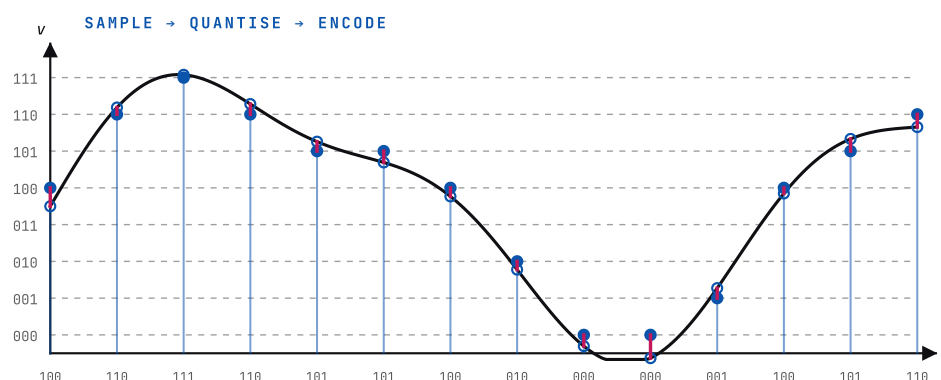
SAMPLING AND QUANTISATION

This is the A/D block opened up, and it is the most numerical-heavy topic on the paper. Three steps turn a smooth wave into bits, and each step has one formula. The chapter settles how fast you must sample, how much error quantising costs, and which of the two SQNR formulas to use — getting that choice wrong is the commonest mistake in the subject.

2.1 The three steps of PCM

- 1 **Sampling** — measure the signal's height at regular times.
- 2 **Quantisation** — round each measured height to the nearest allowed level.
- 3 **Encoding** — write each level number as a binary codeword.

DIAGRAM



the hollow ring is the true sample; the filled dot is the level it was rounded to.
the short pink stub between them is the quantisation error, and it never exceeds half a step.
 $SNR_s = 1.76 + 6.02n$ dB — one more bit is six more decibels, always.

Fig. 2.1 The three steps on one smooth wave. Vertical stems mark the sampling instants, the dashed horizontal lines are the eight allowed levels, and the three-bit word printed under each stem is the level the sample was rounded to. The little gap between the stem tip and the true curve is the quantisation error.

2.2 Nyquist — how often to sample

$f_s \geq 2 f_m$ f_s = sampling rate, f_m = highest frequency in the signal
 $2 f_m$ is called the Nyquist rate

You must take at least two samples in every cycle of the fastest wiggle: one for the peak, one for the trough. Fewer than two and a fast wave can pretend to be a slow one, and you can never tell them apart afterwards. That fault is **aliasing**.

Telephone speech is limited to about 4 kHz, so telephones sample at 8 kHz. Music reaches 20 kHz, so CDs sample at 44.1 kHz — a little above 2×20 kHz, to leave room for the filter.

2.3 Quantisation noise

n	$= \log_2 L$	$L = 2^n$	bits per sample
step	$= V_{pp} / 2^n$		the height of one rung
$e_{\max}$	$= \text{step} / 2$		worst rounding error
P_q	$= \text{step}^2 / 12$		quantisation noise power

Cut the full voltage swing into 2^n equal slices. Because you always round to the nearest level, the worst you can ever be wrong is **half a step**. The step-squared-over-twelve comes from statistics: if the error is equally likely to be anywhere between $-\text{step}/2$ and $+\text{step}/2$, its average power is exactly that. Memorise it; the derivation is not asked.

2.4 SQNR — the two forms, and which to use

Full-scale sine wave only:	$\text{SQNR}_{\text{dB}} = 6.02 n + 1.76$	dB
Whenever V_{rms} is given:	$\text{SNR} = V_{\text{rms}}^2 / (\text{step}^2 / 12)$	then $10 \log_{10}$

THE TRAP

Use **$6.02n + 1.76$** only when the question says the signal is a sine wave filling the whole range. The moment a question hands you an actual V_{rms} value, it wants the **other** formula. Mixing them up is the most common mistake in this topic, and the two answers differ by tens of decibels.

The headline worth carrying into the hall: **every extra bit buys about 6 dB**. Double the number of levels and the quantisation noise drops by 6 dB.

2.5 Worked example — the standard forward problem

WORKED EXAMPLE – TV SIGNAL, 512 LEVELS

A television signal of bandwidth 4.2 MHz is sent by binary PCM with 512 quantisation levels. Find the codeword length, the bit rate, the transmission bandwidth and the SQNR.

given $f_m = 4.2 \text{ MHz}$ $L = 512$

$$\begin{aligned}
 f_s &= 2 f_m = 2 \times 4.2 &&= 8.4 \text{ MSamples/s} \\
 n &= \log_2 512 \quad (512 = 2^9) &&= 9 \text{ bits/sample} \\
 R_b &= n f_s = 9 \times 8.4 \text{e}6 &&= 75.6 \text{ Mb/s} \\
 B_T &= R_b/2 \quad (\text{polar NRZ}) &&= 37.8 \text{ MHz} \\
 B_T &= R_b \quad (\text{RZ}) &&= 75.6 \text{ MHz} \\
 \text{SQNR} &= 6.02(9) + 1.76 = 54.18 + 1.76 &&= 55.94 \text{ dB}
 \end{aligned}$$

No V_{rms} was given, so the full-scale sine assumption holds and the $6.02n$ rule is the right one. Quote **both** bandwidths unless the question names a line code.

2.6 Worked example — the same problem backwards

WORKED EXAMPLE – FITTING PCM INTO A 50 KB/S CHANNEL

A 3.5 kHz signal is sampled, quantised and coded. The channel carries only 50 kb/s. $V_{\text{pp}} = 4 \text{ V}$ and $V_{\text{rms}} = 0.2 \text{ V}$. Find the best SNR obtainable.

$$\begin{aligned}
 f_s &= 2 \times 3.5 \text{ kHz} &&= 7 \text{ kSamples/s} \\
 n_{\text{max}} &= R_b / f_s = 50000 / 7000 &&= 7.142 \\
 n &= 7 \text{ bits/sample} &&\leftarrow \text{ROUND DOWN, the channel is a ceiling} \\
 \text{step} &= V_{\text{pp}} / 2^n = 4 / 128 &&= 0.03125 \text{ V} \\
 P_q &= \text{step}^2/12 = 0.0009766/12 &&= 8.138\text{e-}5 \\
 \text{SNR} &= V_{\text{rms}}^2 / P_q = 0.04 / 8.138\text{e-}5 &&= 491.52 \\
 \text{SNR} &= 10 \log_{10}(491.52) &&= 26.91 \text{ dB}
 \end{aligned}$$

Why only 27 dB here when the TV question gave 56 dB? Because this signal is **quiet**. A full-scale sine across 4 V_{pp} would have $V_{\text{rms}} = 1.414 \text{ V}$; this one has 0.2 V, seven times smaller, so it sits far closer to the quantisation noise floor. That gap is exactly what companding exists to close — Chapter 3.

THE FIVE-MOVE ATTACK PLAN

Almost every PCM numerical is the same five moves in the same order. **1.** f_s from f_m . **2.** n — from L , or from an accuracy requirement, or from R_b/f_s . **3.** the step size from $V_{pp}/2^n$. **4.** $R_b = n f_s$, times N if it is TDM. **5.** SNR by whichever of the two formulas fits.

WHICH WAY TO ROUND n

When n comes from a **channel rate limit**, round **down** — you may not exceed the channel. When n comes from an **accuracy requirement**, round **up** — you must be at least that accurate. Getting this backwards costs the whole question.

CHAPTER 02 AT A GLANCE

- Nyquist: $f_s \geq 2f_m$. Sample slower and a fast wave masquerades as a slow one — aliasing, and it cannot be undone.
- Quantising: $L = 2^n$ levels, $\text{step} = V_{pp}/2^n$, worst error = $\text{step}/2$, noise power $P_q = \text{step}^2/12$.
- SQNR for a **full-scale sine** is $6.02n + 1.76$ dB; when V_{rms} is given, use V_{rms}^2 divided by $\text{step}^2/12$ instead.
- Every extra bit of the codeword buys about **6 dB** of signal-to-quantisation-noise ratio.
- n from a channel rate limit rounds **down**; n from an accuracy requirement rounds **up**.

CHAPTER 03

PCM RATES, COMPANDING AND DELTA MODULATION

Chapter 2 made the bits. This one turns them into a rate and a bandwidth, shares a wire between several signals, and then fixes the complaint that ended Chapter 2 — that a quiet signal gets a terrible SQNR. The fix is companding. Delta modulation, which has appeared on two of the four papers, closes the chapter.

3.1 Bit rate, bandwidth and TDM

$R_b = n f_s$	bits/s, one channel
$R_b = N f_s n$	bits/s, N channels time-division multiplexed
$BT = R_b / 2$	polar NRZ
$BT = R_b$	RZ

Bit rate is samples per second multiplied by bits per sample. Nothing clever. TDM means several signals share one wire by taking turns: signal 1 sends a sample, then signal 2, and so on. With N signals the shared line carries $N \times f_s$ samples per second.

An RZ pulse is only half a bit wide, so it wiggles twice as fast, so it needs twice the bandwidth of NRZ. That is the whole reason for the two bandwidth lines above.

3.2 The example that chains everything together

WORKED EXAMPLE – EIGHT-CHANNEL TELEMETRY

Eight message signals, 2 kHz each, are time-division multiplexed by binary PCM. The sampling error may not exceed 1 percent of the peak amplitude. Sampling is at least 25 percent above the Nyquist rate. Raised-cosine pulses with roll-off 0.2 are used. Find the minimum transmission bandwidth.

$$\begin{aligned}\text{Nyquist rate} &= 2 \times 2 \text{ kHz} &= 4 \text{ kHz} \\ f_s &= 1.25 \times 4 \text{ kHz} &= 5 \text{ kHz per channel}\end{aligned}$$

the 1% rule, for a quantiser covering $-A$ to $+A$ so that step = $2A/L$:

$$\text{step}/2 \leq 0.01 A \rightarrow A/L \leq 0.01 A \rightarrow L \geq 100$$

L must be a power of two: $2^6 = 64$ too few, $2^7 = 128$ enough

$n = 7$ bits/sample $\leftarrow$ ROUND UP, it is an accuracy rule

$$\begin{aligned}\text{total samples/s} &= 8 \times 5000 &= 40\,000 \\ R_b &= 40\,000 \times 7 &= 280 \text{ kb/s} \\ BT &= (R_b/2)(1 + a) = 140k \times 1.2 &= 168 \text{ kHz}\end{aligned}$$

This one question chains sampling, an accuracy-driven bit count, TDM and raised-cosine bandwidth. If you only have time to revise one numerical, revise this one.

3.3 Why uniform quantising is unfair

A uniform quantiser puts its steps at equal spacing all the way up. A loud signal uses all of them and gets a fine SQNR. A quiet signal wanders over two or three steps, so its rounding error is a large fraction of it and the SQNR collapses. Speech is mostly quiet, with rare loud peaks, so this is exactly the wrong behaviour.

a sine at A dB below full scale, uniform quantiser:

$$\text{SQNR}_{\text{dB}} = 6.02n + 1.76 - A$$

so an 8-bit uniform quantiser gives 49.92 dB at full scale, but only 19.92 dB on a signal 30 dB down.

COMPANDING, IN ONE LINE

Compress before quantising, **expand** after reconstructing. The compressor bends the amplitude scale so that small signals are stretched out over many steps and large ones are squeezed into few. Uniform steps on the bent scale are **fine near zero and coarse near full scale** when mapped back — which holds SQNR roughly constant over a wide range of signal levels.

mu-law (North America, $\mu = 255$):

$$y = V_{\max} \cdot \text{sgn}(x) \cdot \ln(1 + \mu|x|/V_{\max}) / \ln(1 + \mu)$$

A-law (Europe, $A = 87.6$):

$$y = A(|x|/V_{\max}) / (1 + \ln A) \quad \text{for } |x|/V_{\max} \leq 1/A$$

$$y = (1 + \ln(A|x|/V_{\max})) / (1 + \ln A) \quad \text{otherwise}$$

mu-law SQNR, large μ : $\text{SQNR}_{\text{dB}} = 6.02n + 4.77 - 20 \log_{10}(\ln(1+\mu))$

for $\mu = 255$, $n = 8$: $48.16 + 4.77 - 14.88 = 38.05 \text{ dB}$, nearly flat

	UNIFORM QUANTISER	COMPANDED QUANTISER
Step spacing	Equal everywhere	Fine near zero, coarse near full scale
SQNR for a loud signal	Best possible	Slightly worse than uniform
SQNR for a quiet signal	Collapses — drops dB for dB with the signal	Holds roughly constant
8 bits gives you	About 50 dB at full scale only	About 38 dB across a wide dynamic range
Used in	Instrumentation, where levels are controlled	Telephony — mu-law and A-law, 8-bit, 64 kb/s

The headline claim to quote: **8-bit companded PCM sounds about as good as 12- to 13-bit uniform PCM** on speech, which is why the telephone network settled on 8 bits at 8 kHz, that is 64 kb/s per voice channel.

3.4 Delta modulation

Delta modulation is PCM stripped down to **one bit per sample**. Instead of coding the sample, code whether the signal went up or down since the last one: send 1 for up, 0 for down, and the staircase at the receiver follows along a step at a time.

to avoid slope overload, for an input $A \sin(2\pi f_m t)$:

$$\text{step} \cdot f_s \geq 2\pi f_m A$$

step too small $\rightarrow$ slope overload (staircase cannot keep up on steep parts)

step too large $\rightarrow$ granular noise (staircase hunts up and down when flat)

WORKED EXAMPLE – MINIMUM STEP SIZE

A 3 V peak, 1 kHz sine is delta modulated at 64 kSamples/s. What is the smallest step that avoids slope overload?

max slope of the input = $2\pi f_m A = 2 \times 3.1416 \times 1000 \times 3 = 18\,850$ V/s

slope the staircase can manage = $\text{step} \times f_s$

$$\text{step} \geq 18\,850 / 64\,000 = 0.2945 \text{ V}$$

Anything smaller and the staircase falls behind on the steep parts of the sine — that is slope overload, and it is a distortion, not a noise. Adaptive delta modulation fixes it by **doubling the step** whenever the predictor cannot keep up, and shrinking it again when it can: step sizes 2, 4, 8, 16 ...

CHAPTER 03 AT A GLANCE

- $R_b = n f_s$ for one channel, $R_b = N f_s n$ for N channels on TDM; $B_T \approx R_b/2$ for NRZ and R_b for RZ.
- A uniform quantiser is unfair to quiet signals: SQNR falls one dB for every dB the signal drops below full scale.
- Companding compresses before quantising and expands after, giving fine steps near zero — mu-law ($\mu = 255$) or A-law ($A = 87.6$).
- 8-bit companded PCM matches roughly 12- to 13-bit uniform PCM on speech; that is the 64 kb/s telephone channel.
- Delta modulation sends one bit per sample. $\text{Step} \times f_s \geq 2\pi f_m A$ or you get slope overload; too large a step gives granular noise.

CHAPTER 04

LINE CODES — THE SIX YOU MUST DRAW

You have bits; now put them on a wire as voltages. A waveform question has appeared on half the papers and it is worth five marks, most of which are for a tidy grid. This chapter gives the rules for the six codes, the four faults a line code exists to prevent, and the two one-word answers the question always ends with.

4.1 Data element against signal element

- **Data element** — the smallest piece of information. That is a bit.
- **Signal element** — the shortest unit, timewise, of the actual signal.
- **r** — how many data elements ride on one signal element.

Think of signal elements as envelopes and data elements as letters. $r = 1$ is one letter per envelope. $r = 2$ stuffs two letters in one envelope: fewer envelopes, less bandwidth, but the levels sit closer together and noise hurts more. $r = \frac{1}{2}$ splits one letter over two envelopes: wasteful, but the extra envelope gives a guaranteed transition for the clock. Manchester is exactly that.

4.2 The four faults a line code must prevent

- 1 **Baseline wandering.** The receiver judges each bit against a running average of the received power. A long run of 1s drags that average up, a long run of 0s drags it down, and once it has moved the receiver compares against the wrong reference.
- 2 **Loss of synchronisation.** If the receiver clock does not tick at the sender rate it samples in the wrong places and reads bits that were never sent.
- 3 **DC components.** A voltage that holds steady puts energy at very low frequencies. Transformer-coupled links and telephone lines cannot pass them, so that part of the signal is simply thrown away.
- 4 **No self-synchronisation.** The timing must be carried inside the data itself, by transitions that mark the start, middle or end of each bit.

THE DESIRABLE-PROPERTIES ANSWER

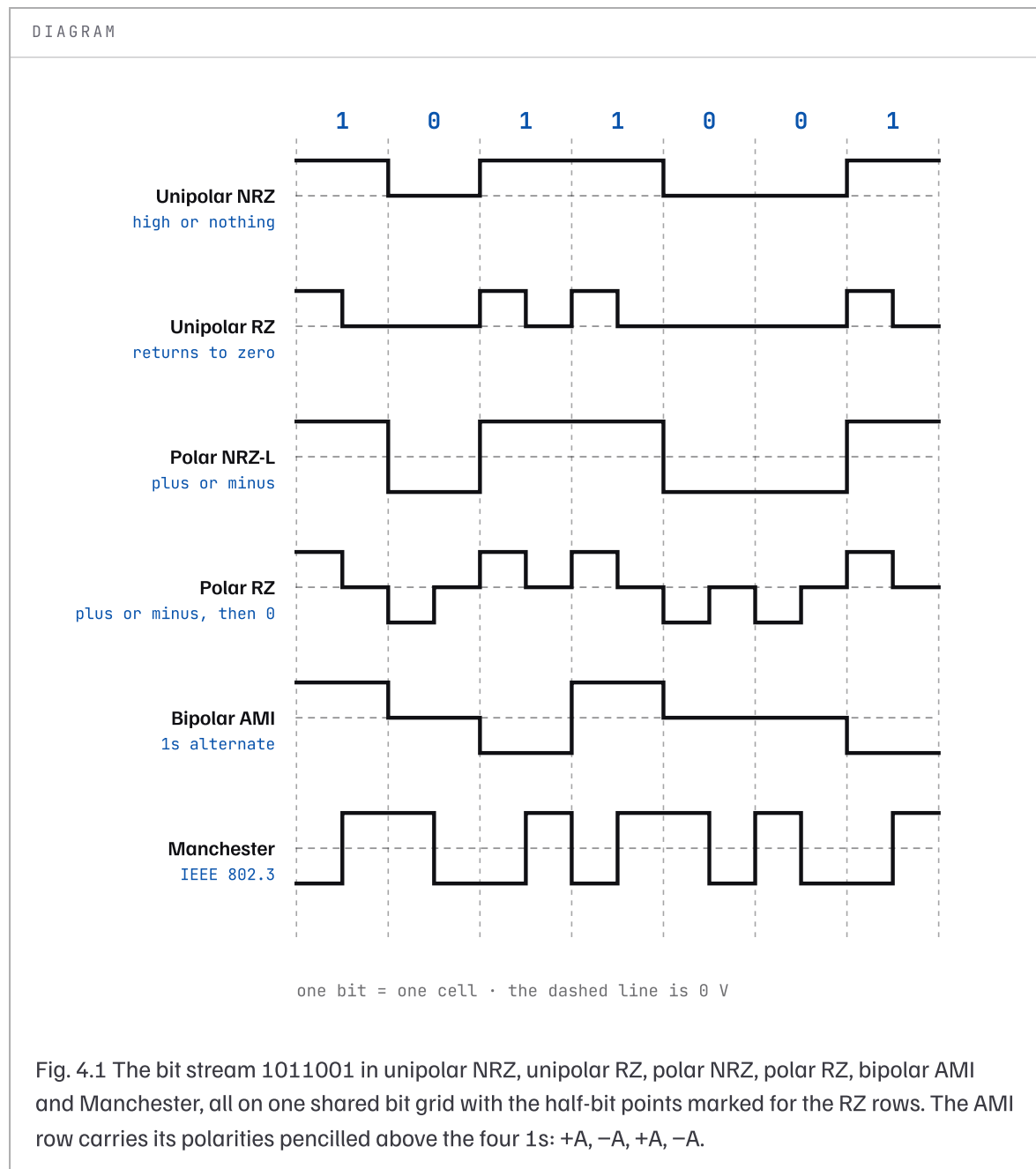
Turn the four faults into requirements and you have a ready-made list-of-six answer: no DC component, no baseline wandering, self-synchronising, small bandwidth, built-in error detection, and low complexity with good noise immunity.

4.3 The rules

CODE	LOGIC 1	LOGIC 0
Unipolar NRZ	+A for the whole bit	0 for the whole bit
Unipolar RZ	+A for half the bit, then 0	0 for the whole bit
Polar NRZ-L	+A for the whole bit	−A for the whole bit
Polar NRZ-I	Invert the level at the start of the bit	Hold the previous level
Polar RZ	+A/2 for half the bit, then 0	−A/2 for half the bit, then 0
Bipolar AMI	Alternates +A, −A, +A, −A ...	0 — no signal at all
Pseudoternary	0 — no signal	Alternates +A, −A, +A ...
Manchester	Mid-bit transition, low to high	Mid-bit transition, high to low
Differential Manchester	No transition at the start of the bit; mid-bit transition always	Transition at the start of the bit; mid-bit transition always

Decoding the names: **NRZ** means the voltage does not return to zero in the middle of the bit. **RZ** means it does. **Unipolar** uses one polarity, **polar** uses two, **bipolar** uses three levels. **AMI** is Alternate Mark Inversion, where “mark” is old telegraph language for a 1. **Pseudoternary** is AMI with the roles of 0 and 1 swapped, nothing more.

4.4 Drawing 1011001



WORKED EXAMPLE – GETTING AMI RIGHT FIRST TIME

For 1 0 1 1 0 0 1, pencil the polarities before you draw anything. Zeros are skipped in the counting; only the 1s alternate.

bit position :	1	2	3	4	5	6	7
bit :	1	0	1	1	0	0	1
is it a 1? :	y	-	y	y	-	-	y
polarity :	+A	0	-A	+A	0	0	-A

So the waveform goes up, flat, down, up, flat, flat, down. Do this line of bookkeeping first and the drawing itself takes ten seconds. Do it in your head and you will drift.

THE TWO ANSWERS THE QUESTION ALWAYS WANTS

Which detects errors? Bipolar AMI. Successive 1s must alternate in polarity, so two +A pulses in a row with no -A between them is an **AMI violation** and an error. No extra bits were spent to get it. **Which synchronises the clock? Manchester.** There is a guaranteed transition in the middle of every bit whatever the data, so the clock is reset every bit period. That is what the doubled bandwidth buys.

WHERE THE MARKS ACTUALLY GO

Rule the bit boundaries **first** and write the bits above them. Mark the half-bit points for the RZ rows. A tidy grid is most of the mark and it is the easiest mark on the paper to drop. Manchester also has two conventions in circulation, so **state which one you are using** in a single line.

CHAPTER 04 AT A GLANCE

- Line codes exist to stop four faults: baseline wandering, loss of synchronisation, DC content, and no self-clocking.
- NRZ holds the level all bit; RZ returns to zero mid-bit; unipolar uses one polarity, polar two, bipolar three levels.
- AMI sends 0 as nothing and alternates the polarity of successive 1s — pencil the polarities before drawing.
- Manchester puts a transition in the middle of every bit, so it is fully self-synchronising, at double the bandwidth.
- AMI detects errors** free, through bipolar violations; **Manchester recovers the clock.** Those are the two one-word answers.

CHAPTER 05

BAUD, BANDWIDTH AND SPECTRA

Bit rate and baud rate are not the same number, and the paper has asked for both from the same waveform. This chapter fixes the difference, derives the Nyquist maximum data rate from it, and then reads the power spectral density plot that answers almost every compare-the-line-codes question in one picture.

5.1 Data rate against signal rate

N = data rate, bits per second (bps)

S = signal rate, signal elements per second (baud)

$S = c \cdot N \cdot (1/r)$ baud

c = case factor, between 0 and 1; take $c = 1/2$ for the average case

r = data elements per signal element

The signal rate is not one fixed number, because it depends on the data pattern. NRZ-I only changes level when a 1 arrives: send all zeros and it never changes at all. So c is a fudge factor saying which case you are in, and unless the question says otherwise it is $1/2$.

WORKED EXAMPLE – BIT RATE AGAINST BAUD RATE

(a) One bit per signal element ($r = 1$), bit rate 100 kb/s. Average baud rate?

$$S = c N (1/r) = 0.5 \times 100\,000 \times 1 = 50\,000 \rightarrow 50 \text{ kbaud}$$

100 000 bits per second travel on only 50 000 signal changes per second, on average.

(b) Two waveforms, bit duration $T_b = 0.5$ ms. Waveform (a) is two-level and one pulse lasts one bit. Waveform (b) is four-level and one pulse lasts two bits.

$$\begin{aligned} \text{(a) pulse rate} &= 1/T_b = 1/0.5\text{e-}3 &= 2000 \text{ baud} \\ \text{bit rate} &= 2000 \times \log_2(2) = 2000 \times 1 &= 2000 \text{ bps} \end{aligned}$$

$$\begin{aligned} \text{(b) pulse width} &= 2 T_b = 1 \text{ ms} \\ \text{pulse rate} &= 1/1\text{e-}3 &= 1000 \text{ baud} \\ \text{bitrate} &= 1000 \times \log_2(4) = 1000 \times 2 &= 2000 \text{ bps} \end{aligned}$$

Both carry the same 2000 bits per second. Waveform (b) does it with half as many pulses, by making each pulse carry two bits — so roughly half the bandwidth. The price is that the four levels sit closer together, so noise flips them more easily. That is multilevel signalling in one example.

5.2 The Nyquist maximum data rate

$$\begin{aligned} N_{\text{max}} &= 2 B \log_2(L) && \text{bits/s} \\ B_{\text{min}} &= S && \text{the minimum bandwidth equals the signal rate} \end{aligned}$$

This is the same statement as $S = cN/r$ wearing different clothes. A signal with L levels carries $\log_2 L$ bits per level, so $r = \log_2 L$; the most signal elements a bandwidth B can carry is $S = B$; and $1/c = 2$ is where the “2” in $2B \log_2 L$ comes from.

So a 3 kHz channel with four levels tops out at $2 \times 3000 \times 2 = 12\,000$ bits per second, and an NRZ-I link at 1 Mb/s has $S = 500$ kbaud and therefore needs at least 500 kHz.

5.3 The spectra

- **NRZ** is at its maximum at $f = 0$. A big value at zero frequency means a big DC component. Its first null is at $fT = 1$, so it is narrow — cheap in spectrum but DC-heavy.
- **Bipolar AMI** is zero at $f = 0$ and humps around $fT \approx 0.5$. No DC, and the bandwidth is no worse than NRZ. Best of both, which is why AMI ran the telephone trunk network.

- **Manchester** is zero at DC too, but the hump sits near $fT \approx 0.75$ and stretches wider. Perfect clock recovery, at roughly double the bandwidth.

SCHEME	R	S AVERAGE	DC?	SELF-CLOCKING?	BANDWIDTH
Unipolar NRZ	1	N/2	Yes, the worst	No	Narrow
Polar NRZ-L / NRZ-I	1	N/2	Yes	No	Narrow
Polar RZ	$\frac{1}{2}$	N	No	Yes	Double
Bipolar AMI	1	N/2	No	Only on the 1s	Narrow
Manchester	$\frac{1}{2}$	N	No	Yes, always	Double
Differential Manchester	$\frac{1}{2}$	N	No	Yes, always	Double

AMI'S ONE WEAKNESS, AND THE CURE

AMI sends a 0 as **nothing at all**, so a long run of zeros is a long silence and the receiver's clock drifts. That is what **scrambling** is for: B8ZS and HDB3 replace a run of zeros with a pattern containing deliberate bipolar violations, which the receiver recognises and strips out. The data rate is unchanged and no extra bits are sent.

WHY NRZ-I RATHER THAN NRZ-L

NRZ-L is absolute: the level tells you the bit. Swap the two wires anywhere in the cable and every bit comes out inverted. NRZ-I is differential: the bit is carried by whether the level **changed**, so swapping the wires changes nothing. That immunity to polarity reversal is why differential schemes are used in practice, and why differential Manchester — polarity-proof and self-clocking — was chosen for Token Ring.

CHAPTER 05 AT A GLANCE

- Data rate N is in bits/s; signal rate S is in baud. $S = cN(1/r)$, and $c = 1/2$ for the average case.
- Bit rate = pulse rate $\times \log_2 L$, so bit rate is never below pulse rate and they are equal only for binary signalling.
- Nyquist maximum data rate: $N_{\max} = 2B \log_2 L$, and the minimum bandwidth equals the signal rate, $B_{\min} = S$.
- PSD: NRZ peaks at DC, AMI is zero at DC and humps near $fT = 0.5$, Manchester is zero at DC but runs about twice as wide.
- AMI sends zeros as silence, so long zero runs break the clock — scramblers (B8ZS, HDB3) inject deliberate bipolar violations to fix it.

CHAPTER 06

ISI AND THE NYQUIST CRITERIA

Every real channel has limited bandwidth, and that single fact causes one disease: intersymbol interference. This chapter is the diagnosis and the cure. It opened the September 2025 paper and it opens our predicted paper, so the four marks here are close to guaranteed.

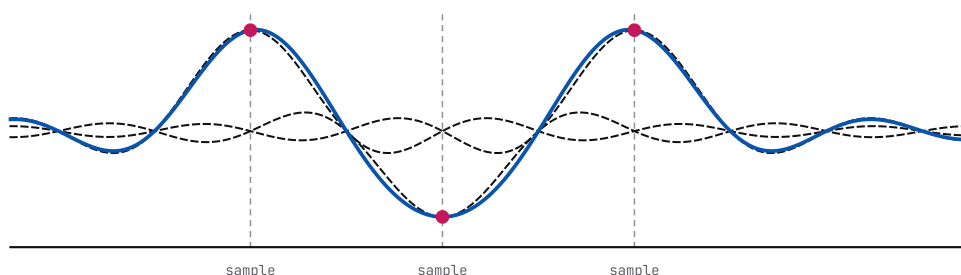
6.1 What ISI is and why it happens

What it is. ISI is the smearing of one transmitted symbol into the time slots of its neighbours, so the value sampled for one bit is contaminated by the tails of the bits around it.

Why it happens. A sharp rectangular pulse contains frequencies out to infinity. Pass it through a channel that removes the high ones and it spreads out in time. The pulse that arrives is wider than the one that was sent, so its tail is still present when the next symbol is sampled.

DIAGRAM

THREE PULSES, AND THE SUM THE RECEIVER ACTUALLY SEES



each pulse has spread past its own bit slot, so at every sampling instant the reading is contaminated by its neighbours. That is intersymbol interference.

Nyquist's first criterion: shape the pulse so it is 1 at its own instant and exactly 0 at every other sampling instant. Then the neighbours contribute nothing, however much they spread.

Fig. 6.1 How ISI makes a wrong bit. Top: three pulses whose tails spread past their own slots, with the overlap shaded. Bottom: the sum, sampled at the middle instant — the tails of the two 1s add under the 0 and push the sample above the threshold, so 101 is read as 111.

THE PICTURE THAT EARNS THE MARK

Shout a word in a large empty hall. The echo of the first word is still bouncing when you say the second, and the listener hears a muddle. ISI is exactly that, in electrical form — and it gets worse the faster you talk, which is why **ISI is what ultimately limits the bit rate.**

ISI IS NOT NOISE

No random hiss was added. The signal interfered with **itself**, purely because the channel was too narrow. So you cannot fix it by shouting louder — turn up the power and the ISI grows in exactly the same proportion. On a quiet wire such as a telephone line, the system is limited by ISI rather than by noise.

6.2 The ISI equation

$$y(t_i) = \underbrace{u \cdot a_i}_{\text{wanted}} + \underbrace{u \cdot \sum_{k \neq i} a_k \cdot p[(i-k)T_b]}_{\text{ISI}} + \underbrace{n(t_i)}_{\text{noise}}$$

Three terms. The first is the bit you actually wanted. The second, with the condition $k \neq i$, is every **other** bit leaking a little of its tail into this sampling instant — that is the ISI. The third is noise. The design goal for the rest of this chapter is to make the middle term exactly zero.

6.3 Nyquist's first criterion

Nyquist does not say stop the pulses overlapping. He accepts that they will. He insists only that **at the exact instants when the receiver looks, the interference happens to be zero**. It is a room full of people talking, where everyone else happens to pause at the moment your friend says each word.

TIME DOMAIN

$$\begin{aligned} p(nT) &= 1 && \text{when } n = 0 \\ p(nT) &= 0 && \text{for every other } n \end{aligned}$$

FREQUENCY DOMAIN (exactly equivalent)

$$\sum \text{over } m \text{ of } P(f + m/T) = T \quad \text{a flat constant}$$

The pulse must be tall at its own sampling instant and exactly zero at every other one. Between those instants it may wander, ring and overshoot as much as it likes — nobody is looking then.

THE FOLDING TEST, DONE WITH SCISSORS

Draw the pulse spectrum $P(f)$. Make copies shifted left and right by the symbol rate $1/T$, $2/T$ and so on. Stack all the copies and add them up. If the total is a **flat horizontal line**, the pulse has zero ISI. Wherever one copy falls, its neighbour must rise by the same amount — that property is called odd symmetry about the band edge, and it is the whole secret.

6.4 Sample rate against bandwidth — three cases

CASE	WHAT HAPPENS
$1/T > 2W$	No pulse can satisfy the condition. Zero-ISI signalling is impossible .
$1/T = 2W$	Exactly one solution: a rectangular spectrum, a sinc pulse in time. The minimum-bandwidth case.
$1/T < 2W$	Many solutions. The usual one is the raised cosine .

the $1/T = 2W$ solution:

$$p(t) = \text{sinc}(\pi t / T)$$

$$P(f) = T \text{ for } |f| < W, \quad 0 \text{ otherwise} \quad \text{a perfect rectangle}$$

minimum (Nyquist) bandwidth: $B_{\min} = R_b / 2 \quad \text{Hz}$

WHY THE PERFECT CASE IS USELESS

Two fatal problems. **One:** a spectrum flat to W then dropping instantly to zero needs a brick-wall filter, which cannot be built. **Two:** the sinc decays only as $1/t$, so a pulse from twenty symbols ago is still contributing — and if the sampling clock is off by a hair, all those slowly-dying tails stop cancelling and the ISI is catastrophic.

6.5 The raised cosine, and the bandwidth formula

The raised cosine is a sinc multiplied by a factor that squashes the tails down fast. You keep the zero crossings that give zero ISI and lose the tails that made the sinc fragile. Its spectrum is flat on top, slopes down the sides as a cosine, then is zero — a cosine curve lifted up so it sits above zero, which is where the name comes from.

$$BT = (R_b / 2)(1 + a) \quad a = \text{roll-off factor}, \quad 0 \leq a \leq 1$$

equivalently $f_B = (1 + a) / (2 T_b)$

$$\text{bandwidth efficiency} \quad B_{\text{eff}} = 2 / (1 + a) \quad \text{bit/s per Hz}$$

$$a = 0 \rightarrow 2 \text{ bit/s/Hz, the impossible brick wall}$$

$$a = 1 \rightarrow 1 \text{ bit/s/Hz, the easiest filter to build}$$

practical systems use $a = 0.2$ to 0.5

Note the letters. Some sources write the roll-off as r and some as α . They are the same thing. Do not let the change confuse you.

WORKED EXAMPLE – FINDING THE ROLL-OFF FACTOR

A 75 kHz channel must carry 0.1 Mb/s of binary data using raised-cosine pulses. Find the roll-off factor.

$$T_b = 1 / R_b = 1 / 0.1 \times 10^6 = 1 \times 10^{-5} \text{ s}$$

$$f_B = 75 \text{ kHz}$$

$$\text{from } f_B = (1 + a) / (2 T_b) :$$

$$1 + a = 2 f_B T_b = 2 \times 75 \times 10^3 \times 1 \times 10^{-5} = 1.5$$

$$a = 0.5$$

Sanity check. The Nyquist minimum for 0.1 Mb/s is 50 kHz. We were handed 75 kHz, which is 1.5 times the minimum, so $a = 0.5$. The bandwidth efficiency is $2/(1+0.5) = 1.33 \text{ bit/s/Hz}$.

The same equation answers all three question types: given R_b and a find B_T ; given R_b and B_T find a ; given B_T and a find the maximum R_b .

6.6 The second criterion, in one paragraph

The first criterion protects the pulse **centres**, so the amplitude you read is right. The second protects the pulse **edges** — the zero crossings — because that is where the receiver gets its timing from. It requires $p(t) = 0.5$ at $t = \pm T/2$. If ISI moves the crossings around, the recovered clock jitters, and a jittery clock samples in the wrong place, undoing everything the first criterion achieved. The raised cosine at $\alpha = 1$ satisfies both.

CHAPTER 06 AT A GLANCE

- ISI is one symbol's tail leaking into its neighbours' sampling instants, caused by the channel being bandwidth limited — it is **not** noise, so more power does not help.
- Nyquist's first criterion: let the pulses overlap, but make $p(nT) = 1$ at $n = 0$ and 0 at every other sampling instant.
- In frequency the same condition is the folding test — shifted copies of $P(f)$ spaced $1/T$ apart must add up to a flat constant.
- The minimum possible bandwidth is $B_{\min} = R_b/2$, reached only by the unbuildable brick-wall sinc pulse.
- Raised cosine: $B_T = (R_b/2)(1 + \alpha)$, efficiency $2/(1 + \alpha)$, with $\alpha = 0.2$ to 0.5 in practice.

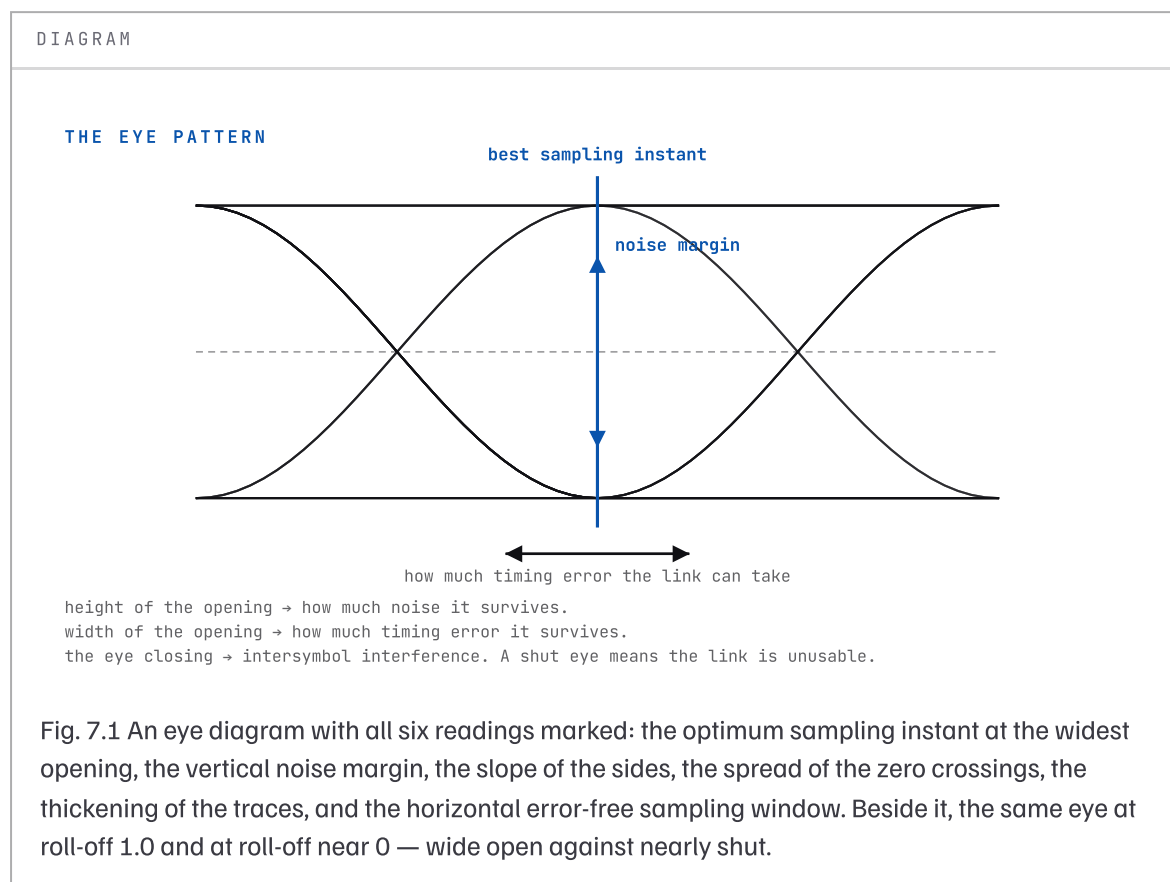
CHAPTER 07

THE EYE, THE EQUALISER AND DUOBINARY

Chapter 6 was theory. This chapter is how the theory is checked, patched and then deliberately inverted. The eye diagram shows ISI on a scope in two seconds; the zero-forcing equaliser cancels what the channel still ruins; and duobinary throws the whole idea into reverse by adding controlled ISI on purpose. Duobinary was a three-mark question in 2025.

7.1 The eye diagram

Take the received waveform, chop it into pieces one symbol long, and lay every piece on top of every other. A clean channel makes every piece follow nearly the same path, leaving a big gap in the middle that looks like an eye. ISI makes the pieces take many paths, the traces spread, and **the eye closes**. On a real scope you do not chop anything — you trigger at the symbol rate and let the traces pile up.



- 1 **Optimum sampling instant** — the point of maximum eye opening. Sample here.
- 2 **Noise margin** — the vertical opening at that point. How much noise you can take before an error.
- 3 **Slope** — how sensitive the link is to sampling time. Steep sides mean a small timing error hurts a lot.
- 4 **Level-crossing ambiguity** — the spread of the zero crossings. This is the timing jitter.

5 Distortion at the sampling point — how much the trace has thickened. This is the ISI.

6 Error-free sampling region — the horizontal width of the eye. How much timing error you can tolerate.

ROLL-OFF	SATISFIES 1ST NYQUIST	SATISFIES 2ND	SAFE SAMPLING WINDOW T_H/T	THE EYE LOOKS
1.0	Yes	Yes	1.00	Wide open and clean
0.8	Yes	No	0.93	Still very good
0.5	Yes	No	0.80	Cluttered, overshoot to ± 1.5
near 0	Yes	No	0.32	Nearly shut, overshoot to ± 3

ONE SENTENCE ON THE EYE

A wide-open eye is a healthy link. Vertically it is losing noise margin; horizontally it is losing timing margin. An eye that has closed completely means the **ISI alone** is enough to cause errors, with no noise required.

Read the table once more and the argument against a tiny roll-off is complete: the safe sampling window collapses from a whole symbol to under a third of one, **and** the waveform swings to ± 3 instead of ± 1 , so the amplifier must handle three times the voltage. The free bandwidth saving is not free.

7.2 The zero-forcing equaliser

A transversal filter is a chain of delay boxes with a multiplier on each tap, all feeding one adder. Feed the received pulse in and choose the $2N + 1$ tap weights so that the output is 1 at the wanted sampling instant and **forced to zero** at the N sampling instants either side. That is the whole idea.

$$p_o[k] = \text{SUM}(n = -N \dots N) \quad c_n \cdot p_r[k - n]$$

zero-forcing condition:

$$p_o[0] = 1$$

$$p_o[k] = 0 \quad \text{for } k = \pm 1 \dots \pm N$$

solve the $(2N+1)$ linear equations: $c = \text{inverse}(Pr) \cdot Po$

WHAT ZERO FORCING COSTS

It cancels ISI at those sampling instants exactly, but it does so by inverting the channel — and where the channel response is small, the inverse is large. So a zero-forcing equaliser **amplifies the noise** in the weak parts of the band. It also only zeroes $2N$ of the neighbours, not all of them.

7.3 Duobinary — controlled ISI on purpose

Also called correlative coding or partial response signalling. Instead of fighting ISI, deliberately introduce a **known** amount of it and subtract it back at the receiver. The relaxed condition is $p(nT) = 1$ for $n = 0$ **and** $n = 1$, zero elsewhere — the interference is limited to the two adjacent samples, and because it is deterministic it can be removed.

```

precoder :  $p(k) = d(k) \text{ XOR } p(k-1), \quad p(-1) = 0$ 
level    :  $1 \rightarrow +1, \quad 0 \rightarrow -1$                                 giving  $a(k)$ 
encoder  :  $y(k) = a(k) + a(k-1)$                                 three levels:  $+2, 0, -2$ 

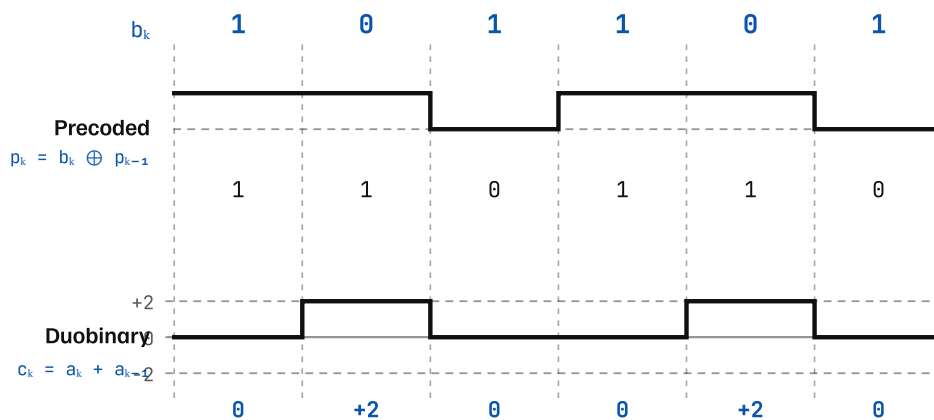
```

decision rule, no memory needed:

$y = 0 \rightarrow d = 1$

$y = +2 \text{ or } -2 \rightarrow d = 0$

DIAGRAM



decode: $b_k = 0$ when $c_k = \pm 2$, and $b_k = 1$ when $c_k = 0$.

the precoder breaks the memory, so one bad symbol cannot poison the ones after it.

Fig. 7.2 The duobinary chain drawn for 101101: the precoder as an XOR with a one-bit delay feeding back, the level converter to ± 1 , the adder that forms $y = a(k) + a(k-1)$, and underneath the three-level output waveform stepping 0, +2, 0, 0, +2, 0 with the decision rule written beside it.

WORKED EXAMPLE — DUOBINARY FOR 1 0 1 1 0 1

Step 1 — precode. A running XOR, starting from $p(-1) = 0$.

```

d : 1  0  1  1  0  1
p : 1  1  0  1  1  0

p0 = 1 xor 0 = 1      p3 = 1 xor 0 = 1
p1 = 0 xor 1 = 1      p4 = 0 xor 1 = 1
p2 = 1 xor 1 = 0      p5 = 1 xor 1 = 0

```

Step 2 — map to polar and add the neighbour. 1 gives +1, 0 gives -1. The starting value $a(-1) = -1$, because $p(-1) = 0$.

```

a :  +1  +1  -1  +1  +1  -1      a(-1) = -1

y0 = +1 + (-1) = 0      y3 = +1 + (-1) = 0
y1 = +1 + (+1) = +2     y4 = +1 + (+1) = +2
y2 = -1 + (+1) = 0      y5 = -1 + (+1) = 0

y :  0  +2  0  0  +2  0

```

Step 3 — decode. The rule needs no memory at all, which is the whole point of having precoded.

```

y :  0  +2  0  0  +2  0
d :  1  0  1  1  0  1      ← the original sequence,
exactly

```

The waveform takes only three levels, -2, 0 and +2, and it is drawn on the same bit grid as any line code.

THE THREE THINGS WORTH ADDING

Why precode. Without it, decoding uses the previous decision, so one error would propagate forever. With it, every symbol is decided from its own sample alone. **Free error detection.** Successive non-zero samples must alternate in polarity when separated only by zeros; two +2s in a row means an error. **The real prize.** Duobinary reaches the theoretical minimum bandwidth $R_b/2$, that is 2 bit/s/Hz, with a **realisable** filter — which the ideal brick wall is not.

CHAPTER 07 AT A GLANCE

- The eye diagram overlays one-symbol slices of the received wave; a wide-open eye is a healthy link, a closed eye means ISI alone causes errors.
- Read six things off an eye: optimum sampling instant, noise margin, slope, zero-crossing jitter, trace thickening, and the error-free sampling width.
- A zero-forcing equaliser is a transversal filter whose taps force the output to zero at the N sampling instants either side — at the cost of amplifying noise.
- Duobinary adds controlled ISI: precode by running XOR, map to ± 1 , then send $y(k) = a(k) + a(k-1)$, giving three levels $+2, 0, -2$.
- Duobinary decoding is memoryless — $y = 0$ means 1, $y = \pm 2$ means 0 — and it hits 2 bit/s/Hz with a buildable filter.

CHAPTER 08

NOISE, THE MATCHED FILTER AND ERROR PROBABILITY

ISI was the signal fighting itself. This chapter brings in the other enemy. It settles which receiver filter is optimal against white noise, proves that the answer depends only on the pulse energy and not its shape, and turns E_b/N_0 into a bit error rate. It also gives the three-line drill for when a question hands you a noise density in dBm instead of an SNR.

8.1 The one ratio that matters

$S = E_b \cdot R_b$ signal power = energy per bit x bits per second

$N = N_0 \cdot B$ noise power = noise density x bandwidth

so $S/N = (E_b/N_0) \cdot (R_b/B)$

That identity is the single most useful line in the topic, and it is faster to derive in the margin than to memorise. S/N depends on how wide your receiver is; E_b/N_0 does not, which is why every error-rate curve in every textbook is plotted against E_b/N_0 and not against SNR.

8.2 The matched filter

A known pulse $s(t)$ of duration T arrives buried in white Gaussian noise. You get to choose one linear filter, and you will sample its output once, at $t = T$. Which filter makes that one sample as far above the noise as possible? The answer is the **matched filter**, and it is unique.

$h(t) = s(T - t)$ for $0 \leq t \leq T$ the pulse, time-reversed

output SNR at $t = T$, maximum possible:

$SNR_{\max} = 2E / N_0$ $E = \text{energy of the pulse} = \int s^2(t) dt$

THE THREE FACTS TO STATE

One. The matched filter maximises the output SNR at the sampling instant, and nothing else can beat it. **Two.** That maximum is $2E/N_0$ — it depends only on the pulse **energy**, not on its shape, so a tall short pulse and a low long one of the same energy do equally well. **Three.** It is exactly equivalent to a correlator: multiply by a stored copy of $s(t)$, integrate over one symbol, sample at T .

WHAT IT DOES NOT DO

A matched filter does **not** preserve the shape of the waveform — its output is the autocorrelation of the pulse, which looks nothing like the pulse. It only guarantees the best possible value at one instant, $t = T$. Sample anywhere else and the guarantee is gone, which is why timing recovery and the matched filter always travel together.

8.3 From E_b/N_0 to bit error rate

antipodal (BPSK, polar NRZ) : $P_e = Q(\sqrt{2 E_b / N_0})$
 orthogonal (coherent FSK) : $P_e = Q(\sqrt{E_b / N_0})$
 on-off (ASK) : $P_e = Q(\sqrt{E_b / N_0})$

a small Q table worth carrying:

$Q(3.0) = 1.35e-3$ $Q(4.0) = 3.17e-5$
 $Q(3.5) = 2.33e-4$ $Q(4.27) = 9.8e-6$
 $Q(4.75) = 1.02e-6$

WORKED EXAMPLE – THE 3 DB THAT SEPARATES BPSK FROM ASK

A link runs at $E_b/N_0 = 9.6$ dB. Find the BER for BPSK, and for ASK at the same E_b/N_0 .

$E_b/N_0 = 10^{(9.6/10)} = 9.120$ (plain ratio)

BPSK : $P_e = Q(\sqrt{2 \times 9.120}) = Q(4.271) = 9.7e-6$ about 1 in 100 000

ASK : $P_e = Q(\sqrt{9.120}) = Q(3.020) = 1.3e-3$ about 1 in 800

To drag ASK down to the same $1e-5$ you need 12.6 dB — exactly **3 dB more power**.

The reason is geometric. ASK puts its two symbols at 0 and A, so they are A apart. BPSK puts them at $-A$ and $+A$, so they are $2A$ apart. Twice the distance means the noise must be twice as large to cause a mistake. **Spend your power moving symbols apart, not turning them off.**

8.4 When the question gives a noise density instead

$$N = N_0 \cdot B \quad \text{watts}$$

$$N_0 = k T \quad k = 1.38e-23 \text{ J/K}$$

at 290 K : $N_0 = 4.0e-21 \text{ W/Hz} = -174 \text{ dBm/Hz}$

in dBm the multiply becomes an add:

$$N_{\text{dBm}} = N_{0\text{dBm/Hz}} + 10 \log_{10}(B)$$

$$P_{\text{dBm}} = 10 \log_{10}(P \text{ in watts}) + 30$$

WORKED EXAMPLE – CAPACITY FROM A NOISE FLOOR

$B = 1 \text{ MHz}$, $N_0 = -174 \text{ dBm/Hz}$, received signal power -100 dBm . Find the channel capacity.

$$\begin{aligned} N &= -174 + 10 \log_{10}(1e6) = -174 + 60 &= -114 \text{ dBm} \\ \text{SNR}_{\text{dB}} &= S_{\text{dBm}} - N_{\text{dBm}} = -100 - (-114) &= 14.0 \text{ dB} \\ S/N &= 10^{(14/10)} &= 25.12 \\ C &= B \log_2(1 + S/N) = 1e6 \times \log_2(26.12) \\ &= 1e6 \times 4.707 &= 4.71 \text{ Mb/s} \end{aligned}$$

Notice step 2. Both powers were already in dBm, so the ratio became a subtraction and nothing had to be converted to watts. Converting first gives the same 25.12 with four more chances to slip a decimal point.

THE THREE-LINE DRILL

Every one of these is the same three lines. **1.** N — from $N_0 B$, or in dBm from $N_0 + 10 \log_{10} B$. If only the temperature is given, start with $N_0 = kT$. **2.** S/N — a subtraction if both are in dBm, then convert with $10^{(\text{dB}/10)^{}}$. **3.** $C = B \log_2(1 + S/N)$. And -174 dBm/Hz is worth memorising outright — it is the thermal noise floor at room temperature.

CHAPTER 08 AT A GLANCE

- $S/N = (E_b/N_0)(R_b/B)$. E_b/N_0 is the fair comparison because it does not depend on receiver bandwidth.
- The matched filter has $h(t) = s(T - t)$ and gives the largest possible output SNR at $t = T$.
- That maximum is $2E/N_0$ — it depends only on the pulse energy, never on its shape, and a correlator does exactly the same job.
- $P_e = Q(\sqrt{2E_b/N_0})$ for BPSK but $Q(\sqrt{E_b/N_0})$ for ASK and coherent FSK — a 3 dB gap in BPSK's favour.
- Noise from a density: $N = N_0B$, or in dBm add $10 \log_{10} B$ to N_0 ; the thermal floor at 290 K is -174 dBm/Hz.

CHAPTER 09

INFORMATION AND ENTROPY

Here the subject becomes beautiful. Shannon asked whether you can put a number on how much information a message carries, and the answer built the digital world. This chapter settles what information means, how to average it into entropy, and how to turn entropy into a bit rate. Entropy or information rate has appeared on three of the four papers.

9.1 Information is surprise

Information depends **only on the probability** of the event. Not on its content, meaning or importance. A message about the least likely event carries the most information.

Two messages sent to you in Surat in June. “It rained today” — it is the monsoon, you expected it, this tells you almost nothing. “It snowed today” — astonishing, you would call everyone you know. Both are four words and cost the same to send. One surprised you.

#	AXIOM	WHY IT MUST HOLD
1	$I(p) \geq 0$	You can never learn a negative amount.
2	$I(1) = 0$	An event that was certain tells you nothing when it happens.
3	$I(p_1 p_2) = I(p_1) + I(p_2)$	Two independent pieces of news should add .
4	$I(p)$ continuous and decreasing in p	Rarer always means more, with no sudden jumps.

Only one function satisfies all four: the logarithm of $1/p$. Axiom 3 is the decisive one, because the logarithm is the function that turns multiplication into addition. If a question says “justify the logarithmic measure of information”, that list is the answer.

9.2 The formula and its units

$$I(x_i) = \log_b(1 / P(x_i)) = -\log_b P(x_i)$$

base 2 → bit the default everywhere in this subject

base e → nat

base 10 → hartley

The minus sign is not making the answer negative. Probabilities are below 1, so their logarithm is negative, and the minus flips it back. Flipping the fraction first, $\log(1/p)$, removes the sign altogether and is easier to type.

LOG₂ ON A CALCULATOR THAT HAS NO LOG₂ BUTTON

$\log_2 a = \ln a / \ln 2 = \log_{10} a / 0.3010$, so **divide the common log by 0.3010**, or multiply by 3.322. Worth knowing outright: $\log_2$ of 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024 is 1 to 10; $\log_2 3 = 1.585$, $\log_2 5 = 2.322$, $\log_2 6 = 2.585$, $\log_2 10 = 3.322$.

WORKED EXAMPLE – ONE EVENT IN THREE UNITS

An event E occurs with probability 0.2. Give I(E) in bits, nats and hartleys.

bits	:	$I = -\log_2(0.2) = \log_2(5) = 2.3219$	bits
nats	:	$I = -\ln(0.2) = \ln(5) = 1.6094$	nats
hartleys	:	$I = -\log_{10}(0.2) = \log_{10}(5) = 0.6990$	hartleys

Same event, same surprise, three units — exactly like measuring one length in metres, feet and inches. Only the base changed.

9.3 Entropy — the average

$H(X) = - \sum P(x_i) \log_2 P(x_i)$ bits per symbol

bounds: $0 \leq H(X) \leq \log_2 m$ $m = \text{size of the alphabet}$

$H = 0$ when one symbol has probability 1

$H = \log_2 m$ when all m symbols are equally likely

Entropy is a **weighted** average. For each symbol, multiply how much information it carries by how often it appears, then add. A very rare symbol carries a lot of information but shows up so seldom it barely moves the average; a common symbol carries little but appears constantly.

DIAGRAM

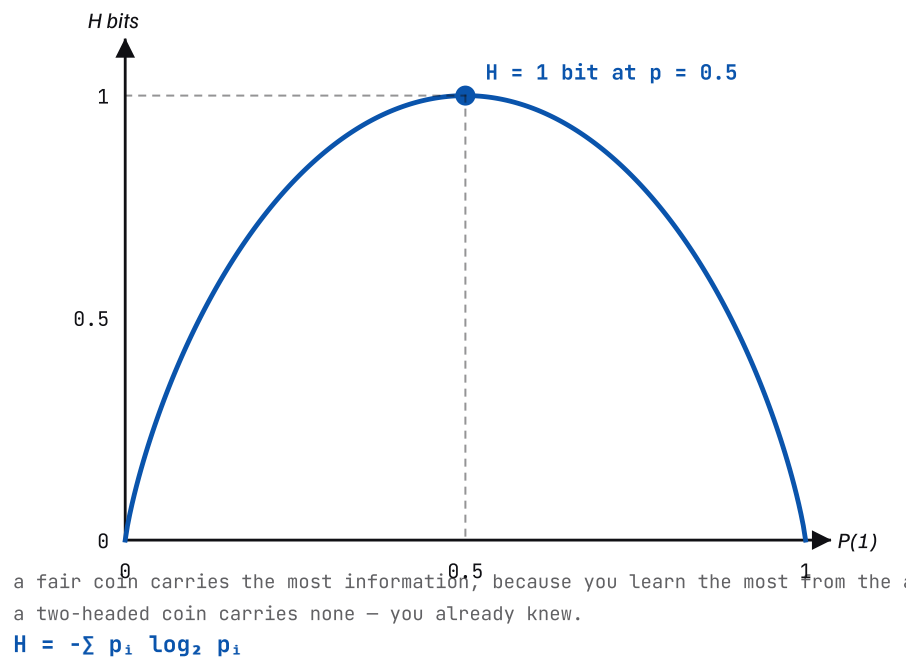


Fig. 9.1 The binary entropy curve, H plotted against $P(1)$ from 0 to 1. It rises to exactly 1 bit at $P = 0.5$ and falls to zero at both ends, where the source always says the same thing and there is nothing left to learn. The curve is symmetric and flat-topped near the middle.

THREE WAYS TO SAY IT, ALL CORRECT IN AN EXAM

Entropy is the **average information per symbol**. Entropy is the **average uncertainty** of the source. Entropy is the **smallest average number of bits** needed to represent one symbol. That last one is the bridge to Chapter 10.

9.4 Information rate

$$R = r \cdot H(X) \quad \text{bits per second}$$

r = symbol rate, symbols per second

H = entropy, bits per symbol

Symbols per second times bits per symbol equals bits per second. Keep **R** and **C** firmly apart: **R** is how fast your source talks, **C** is how fast the channel can listen. Chapter 11 shows everything works when $R < C$ and nothing works when $R > C$.

WORKED EXAMPLE – A FOUR-LEVEL QUANTISED SOURCE

A signal bandlimited to 6 kHz is sampled at the Nyquist rate and quantised into four levels, one symbol each. $P(x_1) = P(x_4) = 1/8$ and $P(x_2) = P(x_3) = 3/8$. Find the information rate. What would it be if the four levels were equally likely?

$$\text{symbol rate : } f_s = 2 \times 6 \text{ kHz} = 12\,000 \text{ symbols/s}$$

$$H = 2 \times (1/8) \log_2(8) + 2 \times (3/8) \log_2(8/3)$$

$$\log_2(8) = 3$$

$$\log_2(8/3) = \log_2(2.6667) = 1.4150$$

$$H = 2(0.125)(3) + 2(0.375)(1.4150)$$

$$= 0.750 + 1.0613 = 1.8113 \text{ bits/symbol}$$

$$R = 12\,000 \times 1.8113 = 21\,735 \text{ bits/s}$$

$$(21.74 \text{ kb/s})$$

$$\text{if equiprobable : } H = \log_2(4) = 2 \text{ bits/symbol}$$

$$R = 12\,000 \times 2 = 24\,000 \text{ bits/s}$$

The 2.27 kb/s difference is the source's **redundancy**. The middle two levels occur three times as often as the outer two, so the sequence is partly predictable and a source coder could exploit it. Sanity check: four symbols means H can never exceed $\log_2 4 = 2$, and 1.81 sits just under.

ENTROPY CHECKLIST

List every symbol with its probability and **confirm they sum to 1** before doing anything else. Watch for probabilities that are neat powers of a half — 0.5, 0.25, 0.125 — because their $\log_2$ values are exactly -1 , -2 , -3 and need no calculator.

Keep four decimals until the end. Check the answer lies between 0 and $\log_2 m$. State the unit: **bits per symbol**.

CHAPTER 09 AT A GLANCE

- Information depends only on probability: $I(x) = -\log_2 P(x)$ bits. Rarer events carry more.
- The logarithm is forced by the axiom that independent news must add, $I(p_1 p_2) = I(p_1) + I(p_2)$.
- Entropy $H(X) = -\sum P \log_2 P$ is the weighted average information per symbol, bounded by $0 \leq H \leq \log_2 m$.
- H is maximum when all symbols are equally likely and zero when one symbol is certain; a fair coin gives exactly 1 bit.
- Information rate $R = r H(X)$ bits per second. Keep R (how fast the source talks) apart from C (how fast the channel listens).

CHAPTER 10

SOURCE CODING — SHANNON–FANO, HUFFMAN, KRAFT

This is the chapter to learn if you learn only one. Source coding has appeared on every single paper we have, and in 2025 it was both methods as two separate five-mark questions — a third of the whole paper. The procedures are mechanical. Learn them until they are automatic, and always finish with the same four numbers plus the Kraft check.

10.1 The four numbers, every time

L	$= \sum P(x_i) \cdot n_i$	average code length, bits/symbol
$H(X)$	$= - \sum P(x_i) \log_2 P(x_i)$	entropy, bits/symbol
η	$= H(X) / L$	efficiency
γ	$= 1 - \eta$	redundancy

source coding theorem: $L \geq H(X)$, so $L_{\min} = H(X)$

THE WHOLE TRICK IN ONE LINE

Give **short codes to common symbols and long codes to rare symbols**. Morse worked this out in 1837: E is the commonest English letter and got a single dot; Q is rare and got four symbols. A fixed-length code wastes bits on the common cases, and the common cases are most of your data.

The source coding theorem is a wall. A source with entropy 2.36 bits/symbol cannot be represented in fewer than 2.36 bits per symbol on average — not by ZIP, not by anything invented in future. But you can get arbitrarily close to it, so a good code lands within a fraction of a percent.

10.2 Classifying a code

PROPERTY	WHAT IT MEANS
Fixed-length	Every codeword has the same number of bits.
Distinct	No two symbols share the same codeword. Fail this and the code is useless.
Prefix-free	No codeword is the beginning of another codeword.
Uniquely decodable	The original message can always be recovered exactly.
Instantaneous	You know a codeword has ended the moment it ends, without peeking ahead.
Optimal	Instantaneous, and with the smallest possible L for those probabilities.

Why prefix-free matters. Take the code 0, 1, 00, 11 and receive the bits 0 0. Is that the third symbol, or the first symbol twice? You cannot tell, because 0 is the prefix of 00. Prefix-free means the instant you have a valid codeword you know it is complete, so you decode bit by bit with no waiting.

THE SUBTLETY THAT CATCHES PEOPLE

Prefix-free is **sufficient** for unique decodability but not **necessary**. The code 0, 01, 011, 0111 is not prefix-free, yet it is uniquely decodable: every codeword starts with a 0 and no codeword contains a 0 anywhere else, so a 0 marks where each one begins. It is not instantaneous though — you must wait for the next 0 before you can commit.

10.3 The Kraft inequality

$$K = \sum 2^{-n_i} \leq 1$$

necessary AND sufficient for an instantaneous binary code with those codeword lengths to exist.

$K = 1$ exactly $\rightarrow$ the code is complete, no branch of the tree wasted.

Think of it as a budget of 1. A codeword of length n costs 2^{-n} of it: a 1-bit codeword costs a half, a 2-bit one a quarter, a 3-bit one an eighth. Short codewords are expensive. Spend 1 or less and the code is buildable; spend more and the codewords would be forced to collide.

WHAT KRAFT DOES NOT TELL YOU

It says an instantaneous code with those lengths **exists**. It does not hand you the codewords, and it does not mean any arbitrary code satisfying it is automatically uniquely decodable. It is an existence test, not a construction method — to actually build the code, use Shannon–Fano or Huffman.

10.4 Shannon–Fano — split from the top

- 1 List the symbols in **decreasing** order of probability.
- 2 Split the list into two groups whose totals are as close to equal as possible. Assign 0 to the upper group, 1 to the lower.
- 3 Repeat the split inside each group until every group holds one symbol.
- 4 Read each codeword by collecting the bits it picked up, left to right.

WORKED EXAMPLE – SHANNON-FANO, THE 2025 SOURCE

$P(A) = 0.35, P(B) = 0.20, P(C) = 0.15, P(D) = 0.15, P(E) = 0.10, P(F) = 0.05.$

running totals : A .35 B .55 C .70 D .85 E .95 F 1.00

candidate splits and their gaps:

after A : 0.35 vs 0.65 gap 0.30

after B : 0.55 vs 0.45 gap 0.10 ← closest, take it

after C : 0.70 vs 0.30 gap 0.40

split 1 : {A,B} = 0.55 → bit 0 {C,D,E,F} = 0.45 → bit 1

split 2a: {A} vs {B} → A = 00, B = 01

split 2b: {C,D} = 0.30 vs {E,F} = 0.15

split 3a: {C} vs {D} → C = 100, D = 101

split 3b: {E} vs {F} → E = 110, F = 111

symbol	P	code	l	P x l
A	0.35	00	2	0.70
B	0.20	01	2	0.40
C	0.15	100	3	0.45
D	0.15	101	3	0.45
E	0.10	110	3	0.30
F	0.05	111	3	0.15

$L = 2.45$ bits/symbol

$$\begin{aligned}
 H(X) &= 0.35(1.5146) + 0.20(2.3219) + 0.15(2.7370) \\
 &\quad + 0.15(2.7370) + 0.10(3.3219) + 0.05(4.3219) \\
 &= 0.5301 + 0.4644 + 0.4106 + 0.4106 + 0.3322 + 0.2161 \\
 &= 2.3639 \text{ bits/symbol}
 \end{aligned}$$

$$\eta = H / L = 2.3639 / 2.45 = 0.9648 \rightarrow 96.48 \%$$

$$\gamma = 1 - \eta = 3.52 \%$$

$$\begin{aligned}
 \text{Kraft} &= 2^{-2} + 2^{-2} + 4 \times 2^{-3} \\
 &= 0.25 + 0.25 + 0.50 = 1.00 \leq 1
 \end{aligned}$$

SATISFIED

$K = 1$ exactly, so the code is **complete** — every leaf position in the binary tree is used and no branch is wasted. That is the best a prefix-free code can do.

10.5 Huffman — merge from the bottom

- 1 List the symbols in decreasing order of probability.
- 2 Add the **two smallest** probabilities together and put the sum back in the list, re-sorting. That is one **reduction**.
- 3 Repeat until only two probabilities remain.
- 4 Assign 0 to one of the last two and 1 to the other, then work backwards: every time you meet a probability you had created by combining, split it and append 0 to one branch and 1 to the other.
- 5 Read each codeword by tracing from the root out to the symbol.

DIAGRAM

HUFFMAN — MERGE FROM THE BOTTOM

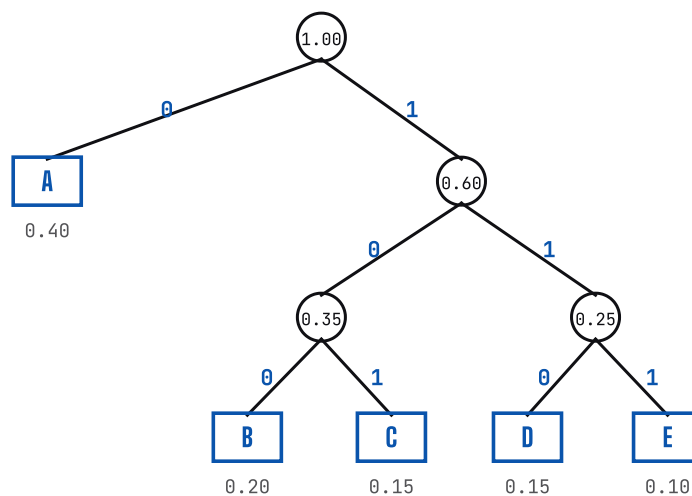


Fig. 10.1 The Huffman tree for 0.35, 0.20, 0.15, 0.15, 0.10, 0.05, built bottom up. The five merges are numbered in the order they were made, the running total sits inside each internal node, every branch carries its 0 or 1, and each leaf shows the codeword read off by walking outwards from the root.

WORKED EXAMPLE – HUFFMAN, THE SAME SOURCE

Step 1 — the reductions. Merge the two smallest, re-sort, repeat.

```

start   : 0.35  0.20  0.15  0.15  0.10  0.05
merge 1 : 0.10 + 0.05 = 0.15 → 0.35  0.20  0.15  0.15  0.15
merge 2 : 0.15 + 0.15 = 0.30 → 0.35  0.30  0.20  0.15
merge 3 : 0.20 + 0.15 = 0.35 → 0.35  0.35  0.30
merge 4 : 0.35 + 0.30 = 0.65 → 0.65  0.35
merge 5 : 0.65 + 0.35 = 1.00 → done

```

symbol	P	code	l	P x l
A	0.35	0	1	0.35
B	0.20	111	3	0.60
C	0.15	100	3	0.45
D	0.15	101	3	0.45
E	0.10	1100	4	0.40
F	0.05	1101	4	0.20

$L = 2.45$ bits/symbol

$H(X) = 2.3639$ bits/symbol (the source has not changed)

$\eta = 2.3639 / 2.45 = 96.48\%$ $\gamma = 3.52\%$

Kraft = $2^{-1} + 3(2^{-3}) + 2(2^{-4})$

$= 0.5 + 0.375 + 0.125 = 1.00 \leq 1$ SATISFIED

The two methods **tie** here: different codewords, identical average length. That happens often with well-behaved probabilities and it does **not** mean the methods are equivalent.

10.6 Where Huffman actually wins

WORKED EXAMPLE – A SOURCE THAT SEPARATES THEM

$P = 0.40, 0.19, 0.16, 0.15, 0.10$. Entropy first, since both parts need it.

$$\begin{aligned} H &= 0.40(1.3219) + 0.19(2.3959) + 0.16(2.6439) \\ &\quad + 0.15(2.7370) + 0.10(3.3219) \\ &= 0.5288 + 0.4552 + 0.4230 + 0.4105 + 0.3322 = 2.1498 \\ &\text{bits/symbol} \end{aligned}$$

SHANNON-FANO

```
{x1} .40 vs .60 gap .20
{x1,x2} .59 vs {x3,x4,x5} .41 gap .18 ← closer, take it
→ x1 = 00, x2 = 01
{x3} .16 vs {x4,x5} .25 gap .09 ← closer
→ x3 = 10, x4 = 110, x5 = 111
lengths 2,2,2,3,3 L = 2.25 eta = 2.1498/2.25 = 95.54 %
```

HUFFMAN

```
merge 1 : 0.15 + 0.10 = 0.25 → 0.40 0.25 0.19 0.16
merge 2 : 0.19 + 0.16 = 0.35 → 0.40 0.35 0.25
merge 3 : 0.35 + 0.25 = 0.60 → 0.60 0.40
merge 4 : 0.60 + 0.40 = 1.00 → done
x1 is merged last, so it lands nearest the root with a 1-bit
codeword
lengths 1,3,3,3,3 L = 2.20 eta = 2.1498/2.20 = 97.72 %
```

Huffman wins by 0.05 bits per symbol, 2.2 percentage points of efficiency. **The structural reason:** Shannon–Fano works top-down and is forced to cut somewhere near the middle, so the 0.40 symbol gets stuck in a two-symbol group and ends up with 2 bits. Huffman works bottom-up, so the most probable symbol is merged **last** and automatically ends nearest the root.

	SHANNON-FANO	HUFFMAN
Direction	Top-down: repeatedly split	Bottom-up: repeatedly merge
Optimal?	Not guaranteed	Always optimal
By hand	Slightly quicker	Slightly longer, but mechanical
On the 2025 source	$L = 2.45, \eta = 96.48\%$	$L = 2.45, \eta = 96.48\%$ — a tie
On the 2022 source	$L = 2.25, \eta = 95.54\%$	$L = 2.20, \eta = 97.72\%$ — Huffman wins

HUFFMAN CODES ARE NOT UNIQUE

Three things are free choices: which branch you call 0, how you break a tie between equal probabilities, and where a merged value is placed among equals when you re-sort. What is **not** free is L . Every correct Huffman code for the same source has the same average length, so if your L matches your friend's you are both right even with different codewords.

FINISH THE ANSWER PROPERLY

Whichever method is asked for, end with the same five things: L , $H(X)$, $\eta = H/L$, **redundancy** $1 - \eta$, and the **Kraft sum**. The 2025 paper asked for the Kraft check explicitly. Writing the codewords alone is half an answer.

CHAPTER 10 AT A GLANCE

- Always report five things: $L = \sum P n$, $H(X)$, $\eta = H/L$, redundancy $1 - \eta$, and the Kraft sum $\sum 2^{-n} \leq 1$.
- The source coding theorem sets a hard floor: $L \geq H(X)$. No code, present or future, beats entropy.
- Prefix-free means no codeword starts another, which makes decoding instantaneous; it is sufficient but not necessary for unique decodability.
- Shannon–Fano splits top-down into near-equal halves; Huffman merges the two smallest bottom-up and is **always** optimal.
- Huffman codewords are not unique, but L always is — matching L means matching answers even with different bits.

CHAPTER 11

CHANNELS AND CHANNEL CAPACITY

Chapter 9 measured the source. This one measures the pipe, and produces the most quoted equation in communications. The binary symmetric channel and a Shannon–Hartley numerical have each appeared on three of the four papers, so this chapter is close to guaranteed marks.

11.1 The channel matrix

$P(y_j | x_i)$ = probability of receiving y_j given that x_i was sent

row rule : SUM over j of $P(y_j | x_i) = 1$ for every row i

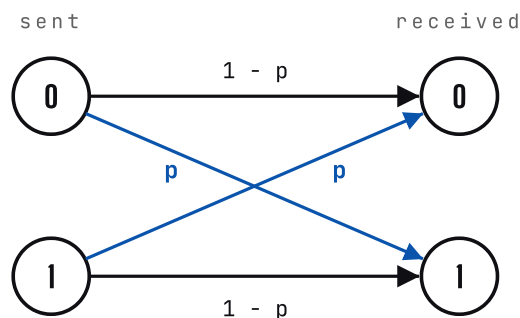
Each row is one transmitted symbol, and reading across it tells you everything that symbol could turn into at the far end. Something must come out — the symbol cannot vanish — so every row sums to exactly 1.

THE COMMONEST MISTAKE WITH CHANNEL MATRICES

Columns do not have to sum to 1. Only rows do. A column is all the ways one particular output could have arisen, and there is no law saying those add to anything in particular.

11.2 The binary symmetric channel

DIAGRAM



$C = 1 - H(p)$ — capacity is 1 at $p = 0$ and at $p = 1$, and 0 at $p = 0.5$

Fig. 11.1 The binary symmetric channel. Two solid horizontal arrows carry a bit through unchanged with probability $1 - p$; two dashed crossing arrows flip it with probability p . Sent values on the left, received on the right — the whole channel is described by that one number p .

$$P(Y|X) = \begin{bmatrix} 1-p & p \\ p & 1-p \end{bmatrix} \quad \text{each row sums to 1}$$

$$C = 1 + p \log_2 p + (1-p) \log_2 (1-p) \quad \text{bits per symbol}$$

which is the same as $C = 1 - H(p)$, $H(p)$ the binary entropy

Read the shape of it: capacity equals **1 minus whatever the noise steals**. The 1 is what a perfect binary channel would give. The two log terms are always negative, so they subtract, and together they are exactly the binary entropy of the noise.

P	WHAT THE CHANNEL IS	CAPACITY
0	Perfect. Nothing is ever flipped.	1 bit/symbol
0.1	Noisy but usable.	0.531 bit/symbol
0.5	The worst possible. Output is a coin toss unrelated to the input.	0
1	Every bit flipped — which is perfect, just invert at the receiver.	1 bit/symbol

That last row is why p is always restricted to $0 \leq p \leq \frac{1}{2}$. Anything past 0.5 is a relabelled version of something below it. The capacity-against- p curve is a U: 1 at both ends, zero in the middle.

WORKED EXAMPLE – BSC CAPACITY AT $P = 0.1$

$$\begin{aligned}
 C &= 1 + p \log_2 p + (1-p) \log_2 (1-p) \\
 &= 1 + 0.1 \log_2(0.1) + 0.9 \log_2(0.9) \\
 &= 1 + 0.1(-3.3219) + 0.9(-0.1520) \\
 &= 1 - 0.3322 - 0.1368 \\
 &= 0.5310 \text{ bits/symbol}
 \end{aligned}$$

One bit in ten is flipped, and it costs you nearly **half** the capacity. Noise is expensive. If the channel runs at r symbols per second, multiply: $C = r C_s$ bits per second.

11.3 The Shannon–Hartley law

$$C = B \log_2(1 + S/N) \quad \text{bits per second}$$

B = bandwidth in Hz

S/N = a PLAIN RATIO, never a dB figure

per sample: $C_s = (1/2) \log_2(1 + S/N)$ bits/sample

Your maximum error-free speed depends on only two things: how much frequency room you have, and how loud you are compared with the noise. Note the asymmetry — **bandwidth sits outside the logarithm and power sits inside**. Doubling B doubles the capacity; doubling S adds only a little. That is why 5G went hunting for millimetre-wave spectrum instead of just shouting louder.

THE DERIVATION IN FOUR LINES

1. The received signal has power $S + N$, so its RMS voltage is $\sqrt{S+N}$, and the noise alone is $\sqrt{N}$. You can only tell apart levels further apart than the noise, so the number of distinguishable levels is $M = \sqrt{1 + S/N}$. **2.** A pulse with M levels carries $\log_2 M = \frac{1}{2} \log_2(1 + S/N)$ bits. **3.** Nyquist says a bandwidth B carries at most $2B$ pulses per second. **4.** Multiply: $C = B \log_2(1 + S/N)$.

NEVER PUT DB STRAIGHT INTO THE FORMULA

Convert first: $S/N = 10^{(dB/10)}$. A question saying 30 dB means a ratio of 1000, not 30. This is the single step students forget, and it destroys the whole answer.

WORKED EXAMPLE – CAN 32 KB/S FIT DOWN A 3.1 KHZ LINE?

Digital voice at 32 kb/s over a 3.1 kHz channel, received SNR 1200. Can it be sent error free? If not, what SNR would be needed, and what is the spectral efficiency?

Step 1 -- what the channel can carry

$$\begin{aligned} C &= 3100 \times \log_2(1 + 1200) = 3100 \times \log_2(1201) \\ &= 3100 \times 10.230 &= 31\,713 \text{ bits/s} \end{aligned}$$

Step 2 -- compare

$$\text{required } R = 32\,000 \text{ b/s} > \text{available } C = 31\,713 \text{ b/s}$$

N_0 -- error-free transmission is impossible. Short by 287 b/s, but short is short.

Step 3 -- the minimum SNR that would work

$$\begin{aligned} 32\,000 &= 3100 \log_2(1 + S/N) \\ \log_2(1 + S/N) &= 32\,000 / 3100 &= 10.3226 \\ 1 + S/N &= 2^{10.3226} &= 1280.6 \\ S/N &= 1279.6 &= 31.07 \text{ dB} \end{aligned}$$

Step 4 -- spectral efficiency

$$\begin{aligned} R / B &= 32\,000 / 3100 &= 10.32 \text{ bit/s per Hz} \end{aligned}$$

Step 5 -- E_b/N_0 for the received data, from $S/N = (E_b/N_0)(R/B)$

$$\begin{aligned} E_b/N_0 &= 1200 / 10.32 &= 116.3 &= 20.65 \text{ dB} \end{aligned}$$

10.32 bit/s/Hz is a demanding figure — it needs roughly 1024-QAM-class signalling — which is exactly why the required SNR is so high. Shannon's theorem is a **wall, not a guideline**: when R exceeds C , no coding scheme whatsoever gives an arbitrarily low error rate.

11.4 Putting R and C together

the condition for a working system: $H(X) \leq R < C$

Shannon's channel coding theorem. If the information rate of a source does not exceed the capacity of the channel, then a coding technique **exists** that makes transmission possible through that unreliable channel with an arbitrarily low error rate.

WHY THAT RESULT IS ASTONISHING

Before 1948 every engineer believed that cutting errors meant slowing towards zero speed. Shannon showed you can run at full speed right up to C and still be near-perfect. You name the error rate; a code exists that beats it. The catch is that he proved such codes **exist** without saying how to build them — finding them took the next fifty years.

And the hard flip side: if $R > C$, reliable communication is impossible no matter how clever the coding. The trade-off that follows is the bandwidth-against-power one: wide bandwidth and low power (spread spectrum, GPS, whose signal arrives weaker than the noise and still works) or narrow bandwidth and high power (a broadcast tower). Both hit the same C .

CHAPTER 11 AT A GLANCE

- A channel matrix has $P(y|x)$ in every cell and **every row sums to 1**; columns need not.
- The BSC is one number p . Its capacity is $C = 1 - p \log_2 p - (1-p) \log_2 (1-p)$, that is 1 minus the binary entropy of the noise.
- BSC extremes: $p = 0$ and $p = 1$ both give 1 bit/symbol; $p = 0.5$ gives zero, because the output is then unrelated to the input.
- Shannon–Hartley: $C = B \log_2(1 + S/N)$. Bandwidth is outside the log and power inside, so bandwidth is far more valuable.
- Convert dB to a plain ratio with $10^{(dB/10)}$ before using the formula, and remember the working condition $H(X) \leq R < C$.

CHAPTER 12

CARRIER MODULATION — ASK, FSK, PSK

Everything so far has been baseband. To go through the air the bits must ride on a carrier, and a sine wave has exactly three things you can change. That is why the list of schemes is so short. The 2025 paper asked for the three waveforms and the BPSK block diagrams as separate questions, which is five marks sitting in one chapter.

12.1 Why a carrier at all

- 1 **Antenna size.** An efficient antenna is about a quarter of a wavelength long. At 1 kHz that is 75 km. At 1 GHz it is 7.5 cm. Raising the frequency is what makes radiating possible at all.
- 2 **Sharing the spectrum.** If everybody transmitted at baseband, every signal would sit on top of every other. Different carriers separate them — that is frequency division multiplexing.
- 3 **The channel.** Many channels will not pass low frequencies. Anything with a transformer or a capacitor in it blocks DC, and a baseband signal is full of it.

The carrier is a plain sine, $A \cos(2\pi f_c t + \phi)$, and it carries no information on its own. Change **A** and you have ASK; change **f_c** and you have FSK; change **φ** and you have PSK. There is no fourth option.

12.2 The three schemes

ASK (on-off keying)

$$s(t) = A \cos(2\pi f_c t) \quad \text{for a 1} \qquad s(t) = 0 \quad \text{for a 0}$$

FSK

$$s(t) = A \cos(2\pi f_1 t) \quad \text{for a 1} \qquad A \cos(2\pi f_2 t) \quad \text{for a 0}$$

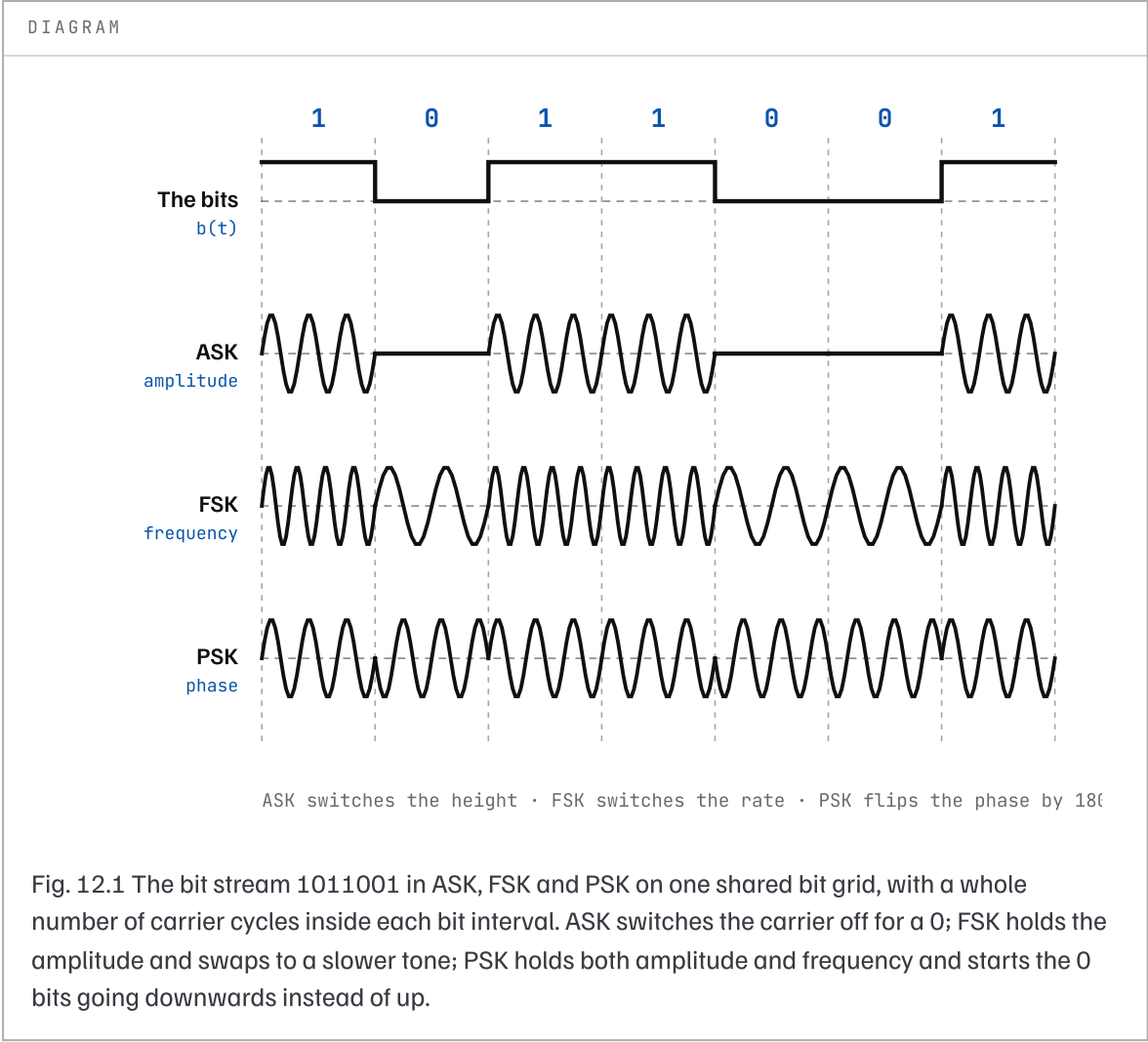
PSK (binary → BPSK)

$$s(t) = A \cos(2\pi f_c t) \quad \text{for a 1}$$

$$s(t) = A \cos(2\pi f_c t + \pi) = -A \cos(2\pi f_c t) \quad \text{for a 0}$$

BPSK IS JUST POLAR NRZ TIMES A CARRIER

Look at that last pair: a 1 gives $+A \cos$, a 0 gives $-A \cos$. That is exactly **polar NRZ multiplied by $\cos(2\pi f_c t)$** . Every BPSK question becomes easy once you see it that way, and it explains why the modulator is nothing but a level converter followed by a multiplier.



	WHAT VARIES	WHAT STAYS FIXED	WHAT A 0 LOOKS LIKE
ASK	Amplitude	Frequency, phase	Carrier switched off — a flat line
FSK	Frequency	Amplitude, phase	The second tone f_2 , drawn with fewer cycles
PSK	Phase	Amplitude, frequency	The same wave inverted — it starts downwards

WHERE THE DRAWING MARKS GO

Rule the bit boundaries **first** and write the bits above them. Put a whole number of carrier cycles in each bit — two or three is plenty. For PSK the waveform must visibly **jump** at every change of bit. Neatness on the grid is most of the mark, and a description in words scores nothing.

12.3 Bandwidth

ASK : $BT = 2 R_b$ null to null; Nyquist minimum is R_b
 PSK : $BT = 2 R_b$ same envelope as ASK, different sign
 FSK : $BT = |f_1 - f_2| + 2 R_b$

with raised-cosine shaping instead of rectangles: $BT = (1 + a) R_b$
 BPSK side frequencies: $f_c \pm R_b/2$

Where the $2R_b$ comes from: the baseband pulse train is R_b wide, measured from 0 up to R_b . Multiplying by a carrier shifts that whole spectrum up to f_c and leaves a mirror image below it, so R_b on one side of zero becomes R_b on **each** side of f_c . FSK is two ASK signals at two carriers, so you need the gap between them plus one width at each end.

WORKED EXAMPLE – BPSK SIDE FREQUENCIES, AND FSK BANDWIDTH

(a) A BPSK modulator has $f_c = 70$ MHz and a bit rate of 10 Mb/s. Find the minimum Nyquist bandwidth and the side frequencies.

the fastest pattern the data can make is 1010101..., one full square-wave cycle every two bits, so the highest fundamental is half the bit rate:

$$f_a = R_b / 2 = 10 / 2 = 5 \text{ MHz}$$

multiplying two sinusoids gives sum and difference:

$$\text{upper side frequency} = f_c + f_a = 70 + 5 = 75 \text{ MHz}$$

$$\text{lower side frequency} = f_c - f_a = 70 - 5 = 65 \text{ MHz}$$

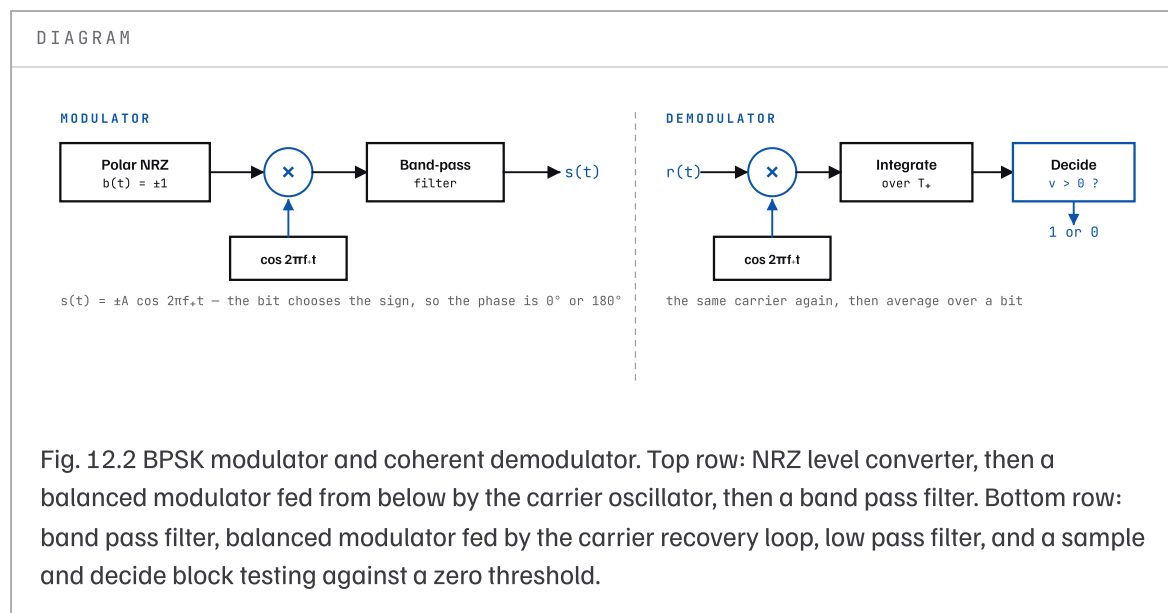
$$B = 75 - 65 = 10 \text{ MHz, which equals } R_b$$

(b) An FSK modulator has $f_1 - f_2 = 100$ kHz at 1 Mb/s with full-width polar pulses.

$$BT = |f_1 - f_2| + 2 R_b = 100 \text{ kHz} + 2 \text{ MHz} = 2.1 \text{ MHz}$$

PSK at the same data rate needs 2 MHz. **FSK always needs more than PSK, by exactly the frequency separation** — that is FSK's main disadvantage. And note (a): the minimum bandwidth of BPSK equals the bit rate. The $2R_b$ above is the null-to-null figure for unfiltered rectangular pulses. Say which one you are quoting.

12.4 The BPSK modulator and demodulator



BLOCK

ITS JOB

NRZ level converter

Turns unipolar data (0 V, +V) into polar (−V, +V). This is what makes the phase flip possible — a 0 must become **negative**, not zero.

Carrier oscillator

Generates the reference sine at f_c .

Balanced modulator (Tx)

A product modulator. Multiplying by +V leaves the carrier alone; by −V inverts it, which **is** the 180° shift. Balanced because the carrier itself is suppressed.

Band pass filter (Tx)

Removes out-of-band products and limits the transmitted spectrum to its channel.

Band pass filter (Rx)

Passes the wanted channel, rejects noise and neighbours, before anything else is done.

Carrier recovery

Regenerates a local carrier locked in frequency **and** phase. BPSK is coherent, so this is essential.

Balanced modulator (Rx)

Multiplies by the recovered carrier. Since $\cos^2\theta = \frac{1}{2}(1 + \cos 2\theta)$, this gives the wanted baseband term plus a component at $2f_c$.

Low pass filter

Throws away the $2f_c$ component and leaves the data.

Sample and decide

Samples once per bit at the best instant and compares with a threshold of zero: positive $\rightarrow$ 1, negative $\rightarrow$ 0.

THE BLOCK PEOPLE LEAVE OUT

Carrier recovery. Leaving it off the receiver diagram is the usual way this question loses marks. It is what makes BPSK coherent, and a 180° lock error inverts every single bit.

12.5 Which is best, and why

	ASK	FSK	PSK (BPSK)
What varies	Amplitude	Frequency	Phase
Bandwidth	$2R_b$	$ f_1 - f_2 + 2R_b$ — widest	$2R_b$
Noise immunity	Worst	Middling	Best
Constant envelope?	No	Yes	Yes
Complexity	Simplest	Middling	Highest — needs a coherent carrier
Error probability	$Q(\sqrt{E_b/N_0})$	$Q(\sqrt{E_b/N_0})$	$Q(\sqrt{2E_b/N_0})$
Used in	Optical fibre, IR remotes, RFID	Modems, Bluetooth, LoRa	Satellite, Wi-Fi, 4G and 5G

THE ONE-LINE ANSWER TO “WHICH IS BEST”

BPSK, because for the same bandwidth as ASK it needs **3 dB less power** for the same bit error rate, and unlike ASK it has a constant envelope so the transmitter amplifier can run in saturation. Its cost is that the receiver must recover the carrier coherently.

WHY ANYONE WOULD USE DPSK INSTEAD

A recovery loop that has just started cannot tell $+\cos$ from $-\cos$, so a coherent BPSK receiver can lock 180° out and decode every bit backwards. The cure is **differential encoding** — send the change rather than the value — which gives DPSK. The price is roughly 3 dB of noise performance, because the reference is itself a noisy received symbol.

CHAPTER 12 AT A GLANCE

- A sine has three changeable properties, so there are three schemes: ASK varies amplitude, FSK frequency, PSK phase.
- BPSK is simply polar NRZ multiplied by the carrier — a 1 sends $+A \cos$, a 0 sends $-A \cos$.
- Bandwidths: ASK and PSK need $2R_b$ null to null; FSK needs $|f_1 - f_2| + 2R_b$, always the widest.
- The BPSK receiver must include **carrier recovery** — without it the link is not coherent, and a 180° lock error inverts every bit.
- BPSK beats ASK and FSK by 3 dB, because its symbols sit $2A$ apart instead of A apart: $P_e = Q(\sqrt{2E_b/N_0})$.

THE WHOLE SUBJECT IN ONE SCREEN

EVERYTHING, CONDENSED

01 THE WHOLE LINK, IN ONE DIAGRAM

- The goal of a digital communication system: send information **reliably** and **efficiently** across a noisy channel.
- The block diagram runs source → A/D → source encoder → channel encoder → modulator → channel → and back the same way in reverse.
- Every receiver block exactly undoes the matching transmitter block — that mirror symmetry is the point of the diagram.
- Source coding removes accidental redundancy; channel coding adds designed redundancy. Not a contradiction — different redundancy.
- Scores: BER, SER, and $\text{SNR}_{\text{dB}} = 10 \log_{10}(E_b/N_0)$. AWGN = additive, white, Gaussian noise, the standard channel model.

02 SAMPLING AND QUANTISATION

- Nyquist: $f_s \geq 2f_m$. Sample slower and a fast wave masquerades as a slow one — aliasing, and it cannot be undone.
- Quantising: $L = 2^n$ levels, $\text{step} = V_{\text{pp}}/2^n$, worst error = $\text{step}/2$, noise power $P_q = \text{step}^2/12$.
- SQNR for a **full-scale sine** is $6.02n + 1.76$ dB; when V_{rms} is given, use V_{rms}^2 divided by $\text{step}^2/12$ instead.
- Every extra bit of the codeword buys about **6 dB** of signal-to-quantisation-noise ratio.
- n from a channel rate limit rounds **down**; n from an accuracy requirement rounds **up**.

03 PCM RATES, COMPANDING AND DELTA MODULATION

- $R_b = n f_s$ for one channel, $R_b = N f_s n$ for N channels on TDM; $B_T \approx R_b/2$ for NRZ and R_b for RZ.
- A uniform quantiser is unfair to quiet signals: SQNR falls one dB for every dB the signal drops below full scale.
- Companding compresses before quantising and expands after, giving fine steps near zero — mu-law ($\mu = 255$) or A-law ($A = 87.6$).
- 8-bit companded PCM matches roughly 12- to 13-bit uniform PCM on speech; that is the 64 kb/s telephone channel.
- Delta modulation sends one bit per sample. $\text{Step} \times f_s \geq 2\pi f_m A$ or you get slope overload; too large a step gives granular noise.

04 LINE CODES — THE SIX YOU MUST DRAW

- Line codes exist to stop four faults: baseline wandering, loss of synchronisation, DC content, and no self-clocking.
- NRZ holds the level all bit; RZ returns to zero mid-bit; unipolar uses one polarity, polar two, bipolar three levels.
- AMI sends 0 as nothing and alternates the polarity of successive 1s — pencil the polarities before drawing.
- Manchester puts a transition in the middle of every bit, so it is fully self-synchronising, at double the bandwidth.
- **AMI detects errors** free, through bipolar violations; **Manchester recovers the clock**. Those are the two one-word answers.

05 BAUD, BANDWIDTH AND SPECTRA

- Data rate N is in bits/s; signal rate S is in baud. $S = cN(1/r)$, and $c = 1/2$ for the average case.
- Bit rate = pulse rate $\times \log_2 L$, so bit rate is never below pulse rate and they are equal only for binary signalling.
- Nyquist maximum data rate: $N_{\text{max}} = 2B \log_2 L$, and the minimum bandwidth equals the signal rate, $B_{\text{min}} = S$.
- PSD: NRZ peaks at DC, AMI is zero at DC and humps near $fT = 0.5$, Manchester is zero at DC but runs about twice as wide.
- AMI sends zeros as silence, so long zero runs break the clock — scramblers (B8ZS, HDB3) inject deliberate bipolar violations to fix it.

06 ISI AND THE NYQUIST CRITERIA

- ISI is one symbol's tail leaking into its neighbours' sampling instants, caused by the channel being bandwidth limited — it is **not** noise, so more power does not help.
- Nyquist's first criterion: let the pulses overlap, but make $p(nT) = 1$ at $n = 0$ and 0 at every other sampling instant.
- In frequency the same condition is the folding test — shifted copies of $P(f)$ spaced $1/T$ apart must add up to a flat constant.
- The minimum possible bandwidth is $B_{\min} = R_b/2$, reached only by the unbuildable brick-wall sinc pulse.
- Raised cosine: $B_T = (R_b/2)(1 + \alpha)$, efficiency $2/(1 + \alpha)$, with $\alpha = 0.2$ to 0.5 in practice.

07 THE EYE, THE EQUALISER AND DUOBINARY

- The eye diagram overlays one-symbol slices of the received wave; a wide-open eye is a healthy link, a closed eye means ISI alone causes errors.
- Read six things off an eye: optimum sampling instant, noise margin, slope, zero-crossing jitter, trace thickening, and the error-free sampling width.
- A zero-forcing equaliser is a transversal filter whose taps force the output to zero at the N sampling instants either side — at the cost of amplifying noise.
- Duobinary adds controlled ISI: precode by running XOR, map to ± 1 , then send $y(k) = a(k) + a(k-1)$, giving three levels $+2, 0, -2$.
- Duobinary decoding is memoryless — $y = 0$ means 1, $y = \pm 2$ means 0 — and it hits 2 bit/s/Hz with a buildable filter.

08 NOISE, THE MATCHED FILTER AND ERROR PROBABILITY

- $S/N = (E_b/N_0)(R_b/B)$. E_b/N_0 is the fair comparison because it does not depend on receiver bandwidth.
- The matched filter has $h(t) = s(T - t)$ and gives the largest possible output SNR at $t = T$.
- That maximum is $2E/N_0$ — it depends only on the pulse energy, never on its shape, and a correlator does exactly the same job.
- $P_e = Q(\sqrt{2E_b/N_0})$ for BPSK but $Q(\sqrt{E_b/N_0})$ for ASK and coherent FSK — a 3 dB gap in BPSK's favour.
- Noise from a density: $N = N_0B$, or in dBm add $10 \log_{10} B$ to N_0 ; the thermal floor at 290 K is -174 dBm/Hz.

09 INFORMATION AND ENTROPY

- Information depends only on probability: $I(x) = -\log_2 P(x)$ bits. Rarer events carry more.
- The logarithm is forced by the axiom that independent news must add, $I(p_1 p_2) = I(p_1) + I(p_2)$.
- Entropy $H(X) = -\sum P \log_2 P$ is the weighted average information per symbol, bounded by $0 \leq H \leq \log_2 m$.
- H is maximum when all symbols are equally likely and zero when one symbol is certain; a fair coin gives exactly 1 bit.
- Information rate $R = r H(X)$ bits per second. Keep R (how fast the source talks) apart from C (how fast the channel listens).

10 SOURCE CODING — SHANNON–FANO, HUFFMAN, KRAFT

- Always report five things: $L = \sum P n$, $H(X)$, $\eta = H/L$, redundancy $1 - \eta$, and the Kraft sum $\sum 2^{-n} \leq 1$.
- The source coding theorem sets a hard floor: $L \geq H(X)$. No code, present or future, beats entropy.
- Prefix-free means no codeword starts another, which makes decoding instantaneous; it is sufficient but not necessary for unique decodability.
- Shannon–Fano splits top-down into near-equal halves; Huffman merges the two smallest bottom-up and is **always** optimal.
- Huffman codewords are not unique, but L always is — matching L means matching answers even with different bits.

11 CHANNELS AND CHANNEL CAPACITY

- A channel matrix has $P(y|x)$ in every cell and **every row sums to 1**; columns need not.
- The BSC is one number p . Its capacity is $C = 1 - p \log_2 p - (1-p) \log_2 (1-p)$, that is 1 minus the binary entropy of the noise.
- BSC extremes: $p = 0$ and $p = 1$ both give 1 bit/symbol; $p = 0.5$ gives zero, because the output is then unrelated to the input.
- Shannon–Hartley: $C = B \log_2 (1 + S/N)$. Bandwidth is outside the log and power inside, so bandwidth is far more valuable.
- Convert dB to a plain ratio with $10^{(dB/10)}$ before using the formula, and remember the working condition $H(X) \leq R < C$.

12 CARRIER MODULATION — ASK, FSK, PSK

- A sine has three changeable properties, so there are three schemes: ASK varies amplitude, FSK frequency, PSK phase.
 - BPSK is simply polar NRZ multiplied by the carrier — a 1 sends $+A \cos$, a 0 sends $-A \cos$.
 - Bandwidths: ASK and PSK need $2R_b$ null to null; FSK needs $|f_1 - f_2| + 2R_b$, always the widest.
 - The BPSK receiver must include **carrier recovery** — without it the link is not coherent, and a 180° lock error inverts every bit.
 - BPSK beats ASK and FSK by 3 dB, because its symbols sit $2A$ apart instead of A apart: $P_e = Q(\sqrt{2E_b/N_0})$.
-

That is the entire revision set — 60 lines. If a line does not make sense, the chapter it came from is one tap away, and the **full course** is behind that.